UNIVERSIDADE ESTADUAL DO MARANHÃO
PRÓ-REITORIA DE GRADUAÇÃO
CENTRO DE CIÊNCIAS TECNOLÓGICAS
NÚCLEO DE TECNOLOGIAS PARA EDUCAÇÃO
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ANDRÉ PEREIRA FERREIRA

ANÁLISE COMPARATIVA DE DESEMPENHO NA ERA DA PERSISTÊNCIA EM MEMÓRIA:

o impacto da arquitetura Worker do FrankenPHP no framework CodeIgniter 4 em contraste com o ecossistema Laravel Octane.

Grajaú - MA
2026

ANDRÉ PEREIRA FERREIRA

ANÁLISE COMPARATIVA DE DESEMPENHO NA ERA DA PERSISTÊNCIA EM MEMÓRIA:

o impacto da arquitetura Worker do FrankenPHP no framework CodeIgniter 4 em contraste com o ecossistema Laravel Octane.

Trabalho de Conclusão de Curso apresentado ao Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Estadual do Maranhão para obtenção do título de Tecnólogo.

Orientador: Me. Paulo Ericson Valentim Silva

Grajaú - MA
2026

ANDRÉ PEREIRA FERREIRA

ANÁLISE COMPARATIVA DE DESEMPENHO NA ERA DA PERSISTÊNCIA EM MEMÓRIA:

o impacto da arquitetura Worker do FrankenPHP no framework CodeIgniter 4 em contraste com o ecossistema Laravel Octane.

Monografia apresentada ao Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da Universidade Estadual do Maranhão para obtenção do título de Tecnólogo.

Aprovado em: ____ / ____ / _____.

BANCA EXAMINADORA

_____
Prof. Me. Paulo Ericson Valentim Silva (Orientador)
Universidade Estadual do Maranhão

_____
[Nome e titulação do(a) examinador(a) 1]
[Instituição]

_____
[Nome e titulação do(a) examinador(a) 2]
[Instituição]

Ao Senhor meu Deus; à minha esposa, Jéssica;
e aos meus familiares, que depositam em mim
uma confiança maior do que julgo merecer.

AGRADECIMENTOS

[Texto de agradecimentos a ser redigido pelo autor.]

RESUMO

Este trabalho analisa o impacto da execução persistente em memória no desempenho do framework CodeIgniter 4 (CI4). Compararam-se o PHP-FPM, o FrankenPHP em modo clássico e o FrankenPHP em modo Worker, com o Laravel Octane como referência de mercado, em contêineres Docker com 1,0 CPU e 512 MB cada, sob carga gerada pelo k6 (100 usuários virtuais, 15 minutos) e monitoramento por Prometheus e Grafana. Em três execuções por cenário, a vazão média foi de 126 requisições por segundo (PHP-FPM), 110 (FrankenPHP clássico), 288 (CI4 em modo Worker) e 369 (Laravel Octane), sem erros. O modo Worker mais que dobrou a vazão do CI4, enquanto o modo clássico não superou o PHP-FPM. Verificou-se ainda que o CI4 mantém o processo vivo, mas reinicia o estado e refaz a conexão com o banco a cada requisição, e que sua memória cresce continuamente durante o teste, ao contrário do Octane. Conclui-se que o modo Worker compensa em vazão para o CI4, exigindo atenção ao consumo de memória.

Palavras-chave: CodeIgniter 4; FrankenPHP; desempenho; benchmarking; PHP.

ABSTRACT

This work analyzes the impact of persistent in-memory execution on the performance of the CodeIgniter 4 (CI4) framework. PHP-FPM, FrankenPHP in classic mode and FrankenPHP in Worker mode were compared, with Laravel Octane as a market reference, in Docker containers limited to 1.0 CPU and 512 MB each, under load generated by k6 (100 virtual users, 15 minutes) and monitored with Prometheus and Grafana. Over three runs per scenario, mean throughput was 126 requests per second (PHP-FPM), 110 (classic FrankenPHP), 288 (CI4 in Worker mode) and 369 (Laravel Octane), with no errors. Worker mode more than doubled CI4 throughput, whereas classic mode did not outperform PHP-FPM. It was also found that CI4 keeps the process alive but resets application state and reconnects to the database on every request, and that its memory grows continuously during the test, unlike Octane. It is concluded that Worker mode pays off in throughput for CI4, requiring attention to memory consumption.

Keywords: CodeIgniter 4; FrankenPHP; performance; benchmarking; PHP.

LISTA DE FIGURAS

Figura 1 – Vazão média (RPS) por cenário, com desvio-padrão das 3 execuções.....19
Figura 2 – Latência p95 por endpoint e cenário.....20
Figura 3 – CPU e RAM médias por cenário.....21
Figura 4 – Painel do Grafana, cenário A (execução de captura).....22
Figura 5 – Painel do Grafana, cenário B (execução de captura).....23
Figura 6 – Painel do Grafana, cenário C (execução de captura).....24
Figura 7 – Painel do Grafana, cenário D (execução de captura).....25

LISTA DE QUADROS

Quadro 1 – Estado preservado entre requisições, por cenário.....18

LISTA DE TABELAS

Tabela 1 – Vazão e latência global por cenário (média das 3 execuções).....19
Tabela 2 – Latência por endpoint (média / p95, em ms).....20
Tabela 3 – Consumo de recursos no patamar sustentado (média das 3 execuções).....21
Tabela F1 – Resultados de cada uma das 12 execuções.....52

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface
CI4	CodeIgniter 4
CPU	Central Processing Unit (unidade central de processamento)
HTTP	HyperText Transfer Protocol
PHP-FPM	PHP FastCGI Process Manager
RAM	Random Access Memory (memória de acesso aleatório)
REST	Representational State Transfer
RPS	Requisições por segundo (requests per second)
VU	Virtual User (usuário virtual)

SUMÁRIO

1 INTRODUÇÃO.....12
1.1 Problema de pesquisa.....13
1.2 Objetivo geral.....13
1.3 Objetivos específicos.....13
2 FUNDAMENTAÇÃO TEÓRICA.....14
2.1 Modelos de execução do PHP.....14
2.2 FrankenPHP: modo clássico e modo Worker.....14
2.3 Laravel Octane e suporte do CodeIgniter 4 ao modo Worker.....14
2.4 Avaliação de desempenho de frameworks web.....15
2.5 Rigor em benchmarks.....15
3 METODOLOGIA.....16
3.1 Objeto de estudo e cenários.....16
3.2 Endpoints de teste.....16
3.3 Instrumentação.....16
3.4 Procedimento de coleta.....17
3.5 Análise dos dados.....17
4 RESULTADOS E DISCUSSÃO.....18
4.1 Validação do ambiente experimental.....18
4.2 Vazão (requisições por segundo).....19
4.3 Latência por endpoint.....20
4.4 Consumo de CPU e memória.....21
4.5 Visão temporal nas execuções de captura.....22
4.6 Discussão.....25
4.7 Ameaças à validade e limitações.....27
5 CONSIDERAÇÕES FINAIS.....28
REFERÊNCIAS.....30
APÊNDICE A — ORQUESTRAÇÃO DOS CONTÊINERES (DOCKER-COMPOSE.YML).....31
APÊNDICE B — CONFIGURAÇÃO DOS SERVIDORES E IMAGENS DOS CENÁRIOS.....35
APÊNDICE C — SCRIPTS DE CARGA DO K6.....40
APÊNDICE D — SCRIPTS DE AUTOMAÇÃO DOS TESTES.....43
APÊNDICE E — CONTROLADOR DOS ENDPOINTS (CODEIGNITER 4).....50
APÊNDICE F — RESULTADOS POR EXECUÇÃO.....52

1 INTRODUÇÃO

A evolução constante das tecnologias web tem exigido que sistemas modernos ofereçam alta escalabilidade e baixo tempo de resposta, forçando desenvolvedores a tomarem decisões estratégicas sobre a arquitetura de execução. Historicamente, a linguagem PHP dominou o cenário do desenvolvimento *server-side* operando no modelo *Request-Response-Die*, no qual o interpretador é iniciado, o *script* executado e todos os recursos destruídos ao final de cada requisição HTTP. Embora o PHP-FPM tenha trazido avanços ao manter processos persistentes, ele ainda opera de forma sequencial e bloqueante, o que gera um *overhead* de *bootstrapping* (inicialização do *framework*) que se repete a cada acesso, limitando o desempenho em cenários de alta carga.

A justificativa para a realização deste *benchmark* reside na necessidade de validar a transição desse modelo tradicional para o paradigma de memória persistente por meio de servidores de aplicação modernos. O surgimento de tecnologias como o Swoole e o FrankenPHP permitiu que o PHP funcionasse com laços de eventos persistentes, mantendo a aplicação carregada na memória RAM e reduzindo a inicialização. O ecossistema Laravel, por meio do Laravel Octane, já consolidou essa prática como padrão de mercado para alto desempenho, tratando nativamente a limpeza de estado e a injeção de dependências.

Entretanto, o *framework* CodeIgniter 4 (CI4), embora amplamente utilizado pela sua leveza e simplicidade, passou a oferecer suporte nativo ao modo Worker do FrankenPHP apenas a partir da versão 4.7. Até então era possível utilizar o modo clássico, mas sem o mesmo desempenho. Para a comunidade de desenvolvimento web, é relevante compreender se adotar o modo Worker no CI4 justifica os ganhos de vazão (RPS) e latência, ou se o uso do FrankenPHP em seu modo clássico já seria suficiente para superar o legado do PHP-FPM.

A relevância deste estudo é multifacetada:

a) eficiência de recursos: em infraestrutura em nuvem, onde o consumo de CPU e RAM impacta diretamente os custos operacionais, validar arquiteturas que otimizam o uso do *hardware* é uma prioridade estratégica;
b) modernização de sistemas legados: muitos sistemas baseados em CodeIgniter precisam escalar sem reescrita completa para outros *frameworks* ou linguagens;
c) adoção de novas ferramentas: o FrankenPHP é uma tecnologia recente que integra o PHP a um servidor escrito em Go, e este estudo fornece dados empíricos sobre seu desempenho em diferentes modos de operação.

1.1 Problema de pesquisa

Em que medida a execução persistente em memória do FrankenPHP (modo Worker) altera a vazão, a latência e o consumo de recursos do CodeIgniter 4, em comparação com o PHP-FPM e com o FrankenPHP em modo clássico, tendo o Laravel Octane como referência?

1.2 Objetivo geral

Comparar o desempenho do CodeIgniter 4 sob PHP-FPM, FrankenPHP em modo clássico e FrankenPHP em modo Worker, utilizando o Laravel Octane como referência, sob carga controlada e com o mesmo orçamento de recursos.

1.3 Objetivos específicos

a) montar um ambiente experimental conteinerizado, reprodutível, com limites idênticos de CPU e memória para os quatro cenários;
b) medir e comparar vazão (requisições por segundo) e latência (p50 e p95) dos cenários sob carga crescente e sustentada;
c) avaliar o consumo de CPU e memória de cada cenário durante o teste;
d) verificar quais elementos (estado da aplicação e conexão com o banco) são efetivamente preservados entre requisições em cada arquitetura;
e) discutir se o ganho observado justifica a complexidade de adoção do modo Worker em aplicações CodeIgniter 4.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Modelos de execução do PHP

No modelo tradicional, o PHP processa cada requisição de forma independente: o interpretador carrega o código, executa-o e libera os recursos ao final, sem preservar estado em memória entre requisições. O PHP-FPM (*FastCGI Process Manager*) mantém um *pool* de processos já iniciados, o que evita criar um processo a cada acesso, mas o código da aplicação, e portanto a inicialização do *framework*, continua sendo executado do zero a cada requisição. Luz, Silva e Emer (2025) descrevem esse modelo como síncrono e sem estado (*stateless*) e o comparam ao modelo assíncrono da extensão Swoole, em experimento de carga com análise estatística (ANOVA e teste t), concluindo que o modo de operação foi o fator determinante do desempenho, com o modelo assíncrono superando o síncrono no atendimento a múltiplas requisições.

2.2 FrankenPHP: modo clássico e modo Worker

O FrankenPHP é um servidor de aplicação para PHP construído sobre o servidor Caddy, que incorpora o interpretador PHP ao próprio processo do servidor (FRANKENPHP, 2026). Sem nenhuma configuração adicional, opera no **modo clássico**, em que funciona como um servidor PHP tradicional, servindo diretamente os arquivos PHP, e pode substituir o PHP-FPM ou o Apache com `mod_php`. O número de conexões aceitas e enfileiradas é limitado apenas pelos recursos do sistema, e o PHP usa um número fixo de *threads* iniciadas na partida, comparável ao modo estático do PHP-FPM (FRANKENPHP, 2026).

No **modo Worker**, o *script* de entrada da aplicação é carregado uma única vez e permanece vivo, atendendo requisições em um laço: a inicialização do *framework* ocorre uma vez por *worker*, e cada requisição executa apenas o tratamento do pedido (FRANKENPHP, 2026). A contrapartida é que o programador passa a ser responsável por não vazar estado de uma requisição para outra, já que objetos e variáveis estáticas podem sobreviver entre elas.

2.3 Laravel Octane e suporte do CodeIgniter 4 ao modo Worker

O Laravel Octane é o pacote oficial do Laravel para executar a aplicação sobre servidores persistentes, entre eles FrankenPHP, Swoole e RoadRunner (LARAVEL, 2026). Ele mantém o *framework* inicializado em memória entre requisições e trata a limpeza do estado por

requisição, o que o torna a referência de mercado para esse modelo no ecossistema PHP. O CodeIgniter 4 passou a oferecer suporte nativo ao modo Worker a partir da versão 4.7, com um *script* de entrada gerado pelo próprio *framework* (CODEIGNITER FOUNDATION, 2026), que reinicia o estado de cada requisição e reconecta recursos como banco de dados e cache.

2.4 Avaliação de desempenho de frameworks web

Estudos recentes comparam *frameworks* web quanto a tempo de resposta, vazão e uso de recursos com ferramentas de carga como o k6. Azzahidi, Wijayanto e Darmawan (2025), por exemplo, avaliaram Spring Boot, Flask, Express.js, Laravel com FrankenPHP e Gin em APIs REST e destacaram o Laravel com FrankenPHP pelo uso eficiente de memória. O k6, mantido pela Grafana Labs, permite descrever cenários de carga em JavaScript, definir rampas de usuários virtuais e exportar métricas para sistemas de monitoramento como o Prometheus (GRAFANA LABS, 2026), e é a ferramenta adotada neste trabalho.

2.5 Rigor em benchmarks

A literatura sobre *benchmarking* ressalta que resultados de desempenho dependem do desenho experimental e do ambiente de execução. Laaber, Basmaci e Salza (2021) tratam da instabilidade de *benchmarks* de *software*, isto é, da variabilidade de medições repetidas, o que justifica repetir as execuções e relatar a dispersão. Zhan (2022) discute a necessidade de *benchmarks* confiáveis, representativos e reprodutíveis para tecnologias computacionais emergentes; Brunst et al. (2022) relatam a experiência de construir e aplicar um conjunto padronizado de *benchmarks* (SPEChpc 2021); e Jia et al. (2013) mostram que a diversidade de aplicações e de volumes de dados afeta as conclusões de um *benchmark*. Essas contribuições orientam as escolhas metodológicas deste estudo: ambiente conteinerizado com limites idênticos de recursos, três execuções por cenário em ordem rotativa, relato de desvio-padrão e discussão explícita das limitações da carga sintética utilizada.

3 METODOLOGIA

A presente pesquisa caracteriza-se como um estudo experimental e quantitativo, de natureza comparativa, que mede o desempenho de diferentes arquiteturas de execução para aplicações PHP em condições controladas.

3.1 Objeto de estudo e cenários

Foram avaliados quatro cenários, todos com o mesmo orçamento de recursos, de 1,0 CPU e 512 MB de RAM por cenário (no cenário A, dividido entre PHP-FPM, com 0,85 CPU e 448 MB, e Nginx, com 0,15 CPU e 64 MB):

- **Cenário A (legado):** CodeIgniter 4.7.3 sobre PHP 8.3-FPM, com 4 processos estáticos e Nginx como servidor web.
- **Cenário B (clássico moderno):** o mesmo aplicativo sobre FrankenPHP em modo clássico, como substituto direto do PHP-FPM.
- **Cenário C (vanguarda):** o mesmo aplicativo sobre FrankenPHP em modo Worker, com 4 *workers* e o suporte nativo do CodeIgniter 4.7 ao modo Worker (arquivo `public/frankenphp-worker.php`, gerado pelo próprio *framework*). Esta versão corrige o texto da etapa anterior, que mencionava uma *bridge* customizada.
- **Cenário D (referência):** Laravel 13 com Laravel Octane sobre FrankenPHP, com 4 *workers*.

Todos usam OPcache com a mesma configuração e o mesmo banco PostgreSQL 16 (contêiner separado, fora do orçamento dos cenários).

3.2 Endpoints de teste

Os quatro cenários expõem os mesmos endpoints: `/health` (resposta mínima, mede a latência base), `/compute` (gera e serializa em JSON 1.000 elementos, com carga de CPU), `/memory` (incrementa um contador estático e retorna o PID, demonstrando persistência de estado) e `/db` (consulta por chave primária a uma tabela com 10.000 linhas, expondo o PID da conexão PostgreSQL).

3.3 Instrumentação

A carga foi gerada pelo Grafana k6 em contêiner isolado (2,0 CPU e 512 MB). Cada execução tem 15 minutos: 2 min de aquecimento até 10 usuários virtuais, 2 min de rampa até

100, 10 min de patamar a 100 usuários e 1 min de desaceleração. Cada usuário dispara requisições continuamente, sem espera entre elas, com a distribuição 10% `/health`, 30% `/compute`, 30% `/memory` e 30% `/db`. As métricas foram exportadas ao Prometheus, visualizadas no Grafana, e o consumo de CPU e RAM dos contêineres foi amostrado com `docker stats` a cada cerca de 7 segundos.

3.4 Procedimento de coleta

Cada cenário foi executado três vezes, em ordem rotativa (A, B, C, D, A, B, C, D, A, B, C, D), para reduzir o viés de ordem e de temperatura do hospedeiro. Antes de cada execução, os contêineres anteriores eram derrubados e as métricas do Prometheus, apagadas. Uma execução do cenário D (terceira rodada) foi refeita isoladamente após falha de rede no *build*. Os experimentos rodaram em um notebook com processador AMD Ryzen 7 5700U (8 núcleos e 16 *threads*) e 14 GiB de RAM, com Linux e Docker 28.1.1; a limpeza do cache do sistema operacional entre execuções não foi possível por falta de permissão administrativa.

3.5 Análise dos dados

Para cada cenário foram calculadas a média e o desvio-padrão, entre as três execuções, da vazão (RPS), da latência (p50 e p95), da taxa de erros e do consumo de CPU e RAM no patamar sustentado. Os ganhos foram expressos em variação percentual sobre o cenário A. Considerou-se relevante uma diferença muito maior que o desvio-padrão entre execuções. O processamento é feito pelo script `scripts/analyze.py` (Apêndice D), que gera as tabelas e figuras do capítulo de Resultados.

4 RESULTADOS E DISCUSSÃO

Este capítulo apresenta os resultados obtidos nos quatro cenários descritos na Metodologia e os interpreta à luz do referencial teórico. Os valores oficiais provêm de 12 execuções do teste de carga (3 repetições por cenário, sem navegador ou outra ferramenta concorrendo pelos recursos do hospedeiro). As figuras extraídas do Grafana (Figuras 4 a 7) vêm de uma rodada extra, feita apenas para captura de tela, e não entram nas médias.

4.1 Validação do ambiente experimental

Antes das medições, cada cenário foi submetido a um teste de sanidade (*smoke test*) e os quatro endpoints (`/health`, `/compute`, `/memory` e `/db`) responderam com sucesso. Nas 12 execuções válidas, todas as verificações (*checks*) do k6 passaram e a taxa de falhas HTTP foi de 0,00%, o que indica que as diferenças de desempenho adiante não decorrem de erros de aplicação.

Os endpoints `/memory` e `/db` revelam o que cada arquitetura realmente preserva entre requisições. Em verificação manual, três requisições consecutivas a cada cenário resultaram em:

Quadro 1 – Estado preservado entre requisições, por cenário

Cenário	**`/memory`: contador e PID**	**`/db`: PID da conexão PostgreSQL**
A – CI4 + PHP-FPM	contador 1, 1, 1; PIDs diferentes (8, 7, 9)	muda a cada requisição (nova conexão)
B – CI4 + FrankenPHP clássico	contador 1, 1; PID 1	não verificado
C – CI4 + FrankenPHP Worker	contador 1, 1, 1; PID fixo (1)	muda a cada requisição (62, 63, 64)
D – Laravel Octane	contador 1, 2, 3; PID fixo (14)	fixo (69): conexão reutilizada

Fonte: elaborado pelo autor (2026), por verificação manual com `curl`.

O resultado é relevante: apenas o Laravel Octane (D) preservou tanto o estado da aplicação quanto a conexão com o banco. No CodeIgniter 4 em modo Worker (C), o processo PHP permanece vivo (PID fixo), mas o contador é reiniciado e a conexão com o banco é refeita a cada requisição, ou seja, o *framework* reinicia o estado da aplicação entre requisições por projeto (como medida de isolamento) e fecha a conexão ao final. Portanto, o ganho do cenário C vem da eliminação da inicialização do *framework* (carregamento de classes, serviços e configuração), e não do reaproveitamento de estado ou de conexão. Esse achado corrige a

expectativa inicial do trabalho, descrita na Introdução e nos materiais do projeto, de que o contador do `/memory` cresceria no cenário C.

4.2 Vazão (requisições por segundo)

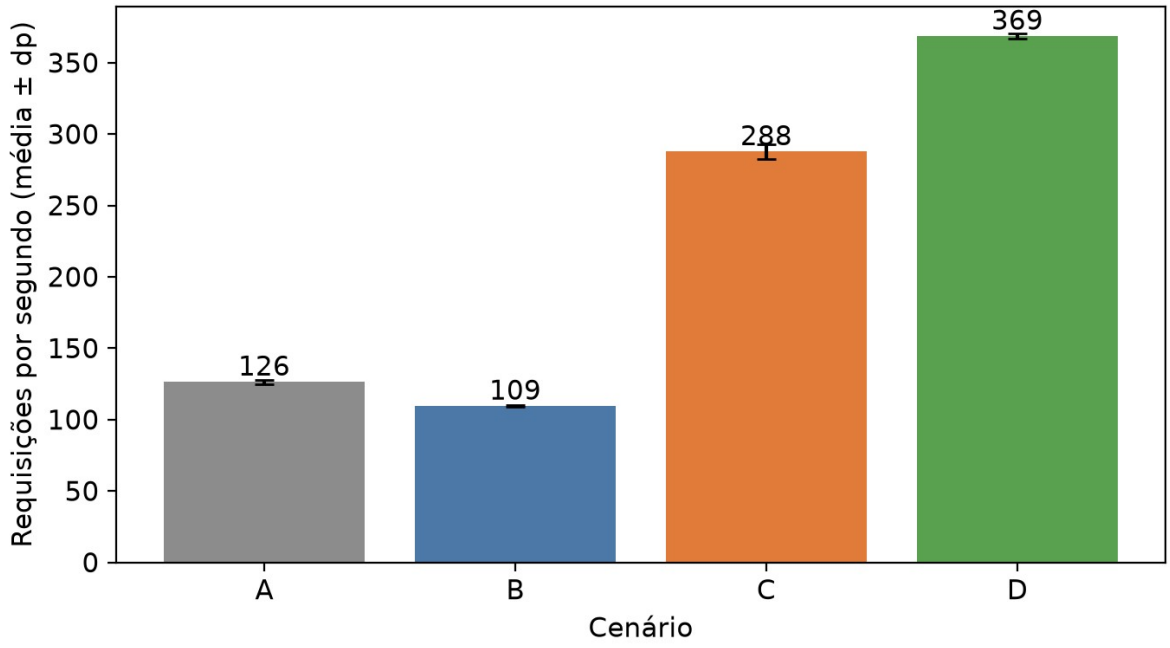
A Tabela 1 e a Figura 1 apresentam a vazão média e o desvio-padrão das três execuções de cada cenário, sob 100 usuários virtuais simultâneos durante o patamar sustentado do teste, em um orçamento de 1,0 CPU e 512 MB por cenário.

Tabela 1 – Vazão e latência global por cenário (média das 3 execuções)

Cenário	RPS (média ± dp)	Latência p50 (ms)	Latência p95 (ms)	Erros (%)
A – CI4 + PHP-FPM	126,4 ± 1,4	764,5	890,9	0,00
B – CI4 + FrankenPHP clássico	109,5 ± 0,5	877,7	1002,2	0,00
C – CI4 + FrankenPHP Worker	287,7 ± 5,1	307,9	430,8	0,00
D – Laravel Octane	368,7 ± 2,1	275,8	314,8	0,00

Fonte: elaborado pelo autor (2026), a partir dos resumos do k6. Nota: p50 e p95 calculados sobre toda a execução, incluindo as rampas.

Figura 1 – Vazão média (RPS) por cenário, com desvio-padrão das 3 execuções



Fonte: elaborado pelo autor (2026).

Em relação ao cenário A (referência), o modo clássico do FrankenPHP (B) apresentou vazão 13,4% menor, o Worker com CodeIgniter 4 (C) apresentou vazão 127,7% maior (cerca de 2,3 vezes) e o Laravel Octane (D) apresentou vazão 191,7% maior (cerca de 2,9 vezes). Entre C e B, a diferença é de 162,8% a favor de C. Todas essas diferenças são muito maiores

que a variação entre execuções do mesmo cenário (desvio-padrão entre 0,4% e 1,8% da média), o que dá segurança de que não são ruído de medição.

4.3 Latência por endpoint

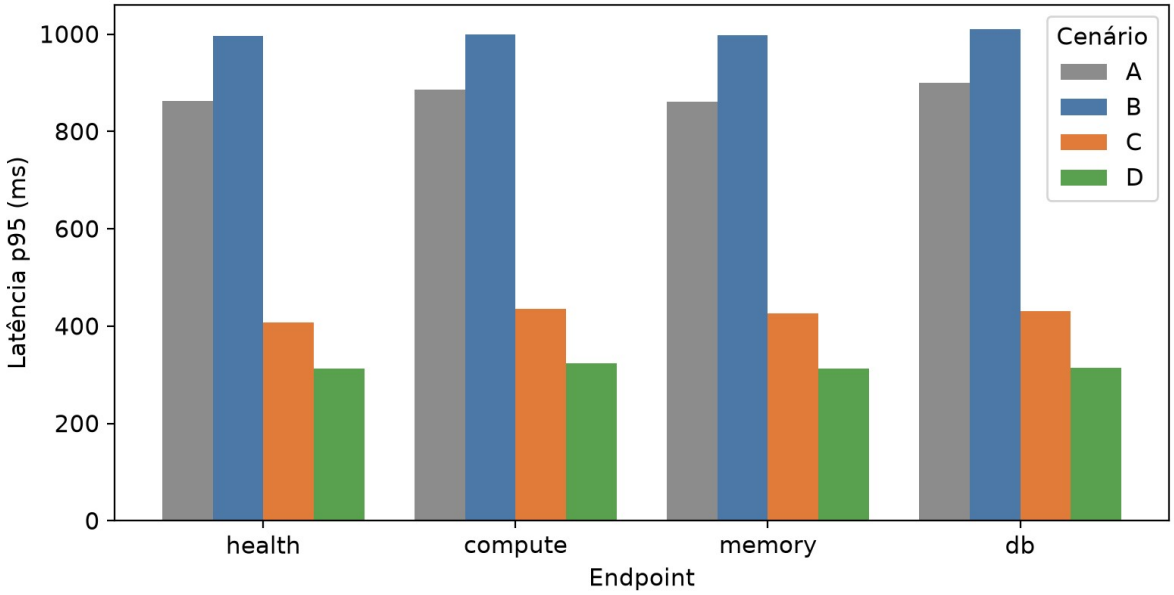
A Tabela 2 e a Figura 2 detalham a latência média e o percentil 95 por endpoint.

Tabela 2 – Latência por endpoint (média / p95, em ms)

Cenário	/health	/compute	/memory	/db
A – CI4 + PHP-FPM	599,9 / 862,2	609,2 / 885,7	600,8 / 861,8	644,8 / 899,7
B – CI4 + FrankenPHP clássico	708,6 / 996,2	709,0 / 1000,0	707,2 / 997,5	719,8 / 1009,6
C – CI4 + FrankenPHP Worker	265,9 / 407,4	277,3 / 435,4	266,6 / 425,5	268,7 / 431,0
D – Laravel Octane	207,4 / 312,5	217,0 / 324,4	207,3 / 312,4	209,5 / 313,7

Fonte: elaborado pelo autor (2026).

Figura 2 – Latência p95 por endpoint e cenário



Fonte: elaborado pelo autor (2026).

Um achado importante é que a latência é praticamente a mesma nos quatro endpoints dentro de cada cenário, embora o `/health` não faça processamento e o `/compute` gere e serialize mil registros. A explicação é a forma de carga: o k6 mantém 100 usuários virtuais disparando requisições continuamente, sem tempo de espera entre elas, e o servidor, limitado a 1 CPU, está saturado (Seção 4.4). Nessa condição, o tempo de resposta é dominado pelo tempo de fila e, pela Lei de Little, aproxima-se de usuários ÷ vazão. A conta confere: 100 ÷ 126,4 ≈ 791 ms no cenário A (p50 medido: 765 ms) e 100 ÷ 368,7 ≈ 271 ms no cenário D (p50 medido: 276 ms). Consequentemente, a latência aqui é um espelho da vazão, e não uma medida

independente do custo de cada endpoint. Isso limita a conclusão sobre o diferencial do `/db` (reaproveitamento de conexão): a diferença entre `/db` e `/health` é pequena em todos os cenários (por exemplo, 45 ms no cenário A e 3 ms no cenário C), mas o desenho do experimento não isola o custo de cada endpoint com precisão. Para isso seria necessária carga com taxa de chegada fixa (modelo aberto), tema indicado nos trabalhos futuros.

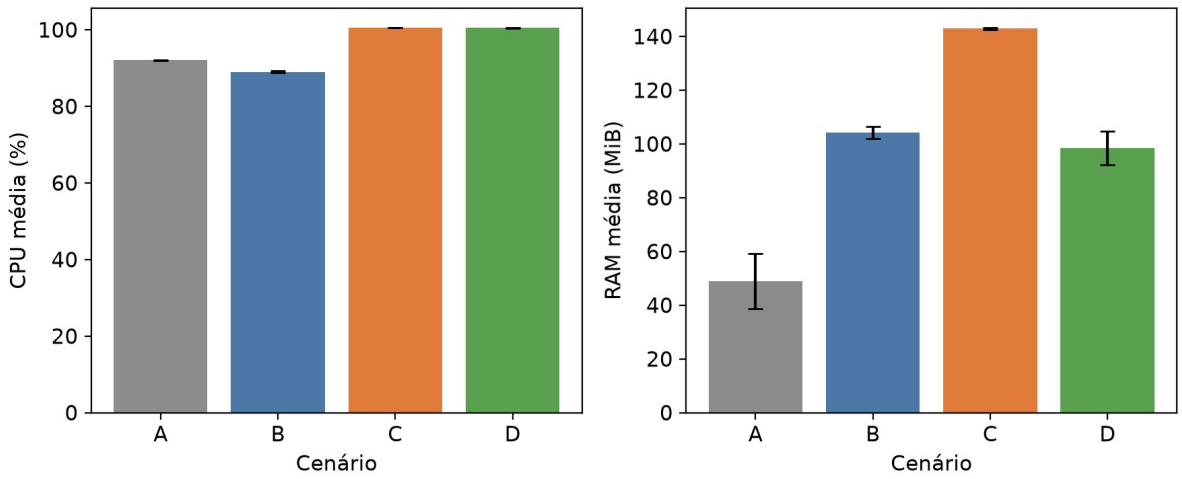
4.4 Consumo de CPU e memória

Tabela 3 – Consumo de recursos no patamar sustentado (média das 3 execuções)

Cenário	CPU média (% de 1 núcleo)	RAM média (MiB)	RAM pico (MiB)
A – CI4 + PHP-FPM (app + Nginx)	92,1 ± 0,1	48,9 ± 10,3	50,1
B – CI4 + FrankenPHP clássico	89,0 ± 0,3	104,2 ± 2,2	114,6
C – CI4 + FrankenPHP Worker	100,6 ± 0,1	142,9 ± 0,4	197,4
D – Laravel Octane	100,5 ± 0,1	98,5 ± 6,2	104,3

Fonte: elaborado pelo autor (2026), a partir de amostras de `docker stats` a cada cerca de 7 s. Nota: no cenário A, soma de PHP-FPM e Nginx.

Figura 3 – CPU e RAM médias por cenário



Fonte: elaborado pelo autor (2026).

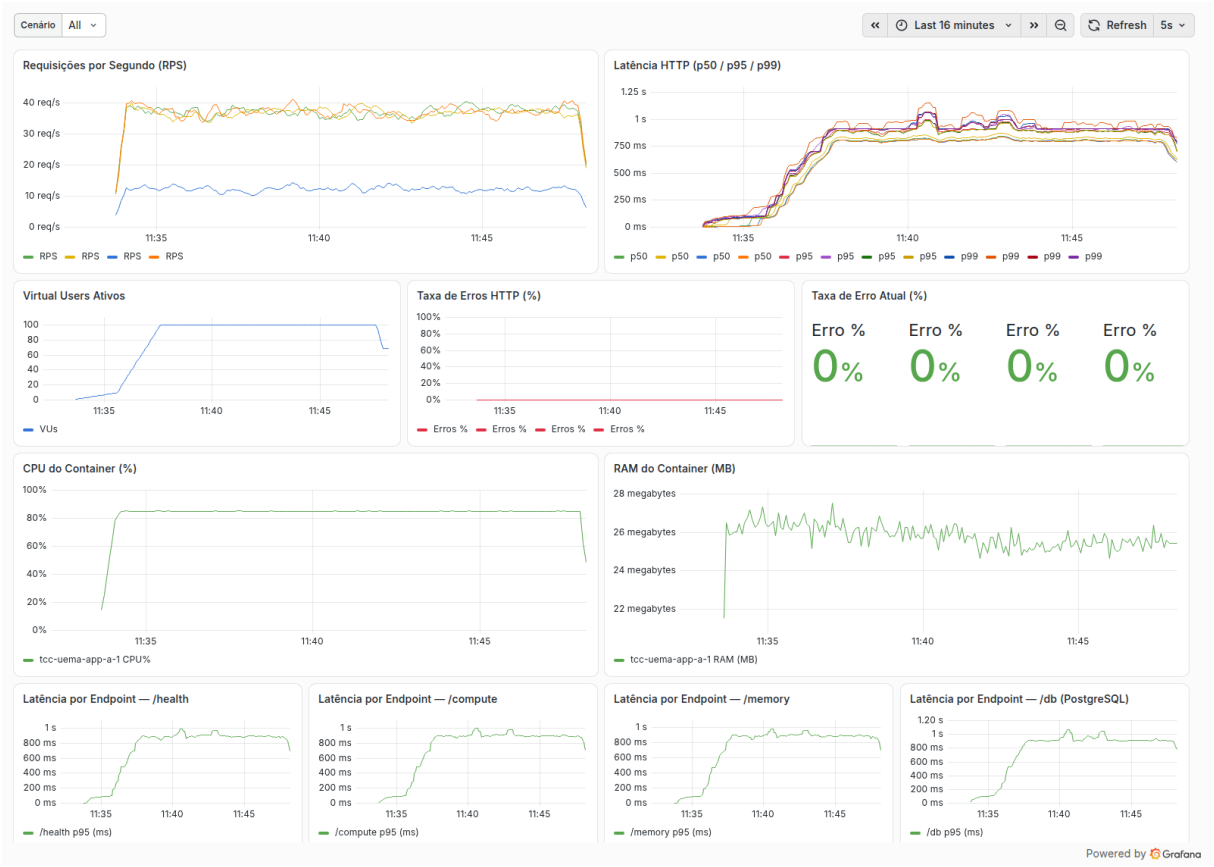
Em todos os cenários a CPU ficou próxima do limite de 1,0 núcleo (89% a 101%), confirmando a saturação e que o gargalo do experimento é a CPU, e não a memória. Os cenários persistentes (C e D) chegaram a 100% de uso porque processaram mais requisições com a mesma CPU; ou seja, converteram o recurso em trabalho útil com mais eficiência, e não consumiram mais por requisição. O PHP-FPM foi o mais econômico em memória (49 MiB), custo esperado de manter a aplicação carregada em memória nos demais.

O comportamento da memória ao longo do tempo diferencia os cenários persistentes (Figuras 6 e 7). No Laravel Octane (D), a RAM se estabiliza em cerca de 100 MiB logo no início do patamar. No CodeIgniter 4 em modo Worker (C), a RAM cresce de forma contínua e aproximadamente linear durante toda a execução, de cerca de 45 MiB para cerca de 210 MiB ao final dos 15 minutos, sem formar patamar. Em testes mais longos, esse crescimento poderia atingir o limite de 512 MiB; ele indica acúmulo de memória entre requisições no processo persistente (por exemplo, objetos ou registros mantidos pelo *framework* ou pela aplicação), o que o presente experimento não investigou. A média de 143 MiB da Tabela 3 reflete, portanto, um valor que ainda crescia, e o pico (197 MiB) não é um teto.

4.5 Visão temporal nas execuções de captura

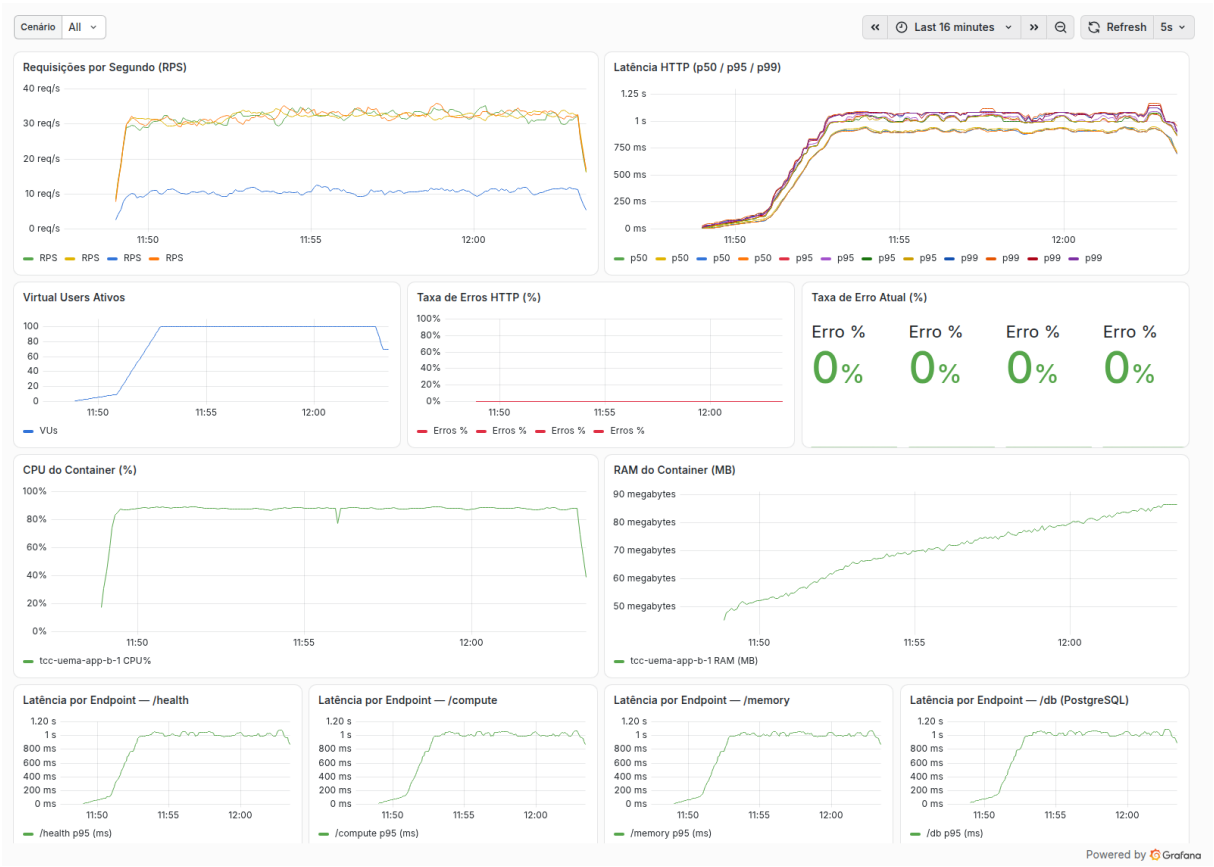
As Figuras 4 a 7 mostram o painel do Grafana de cada cenário em uma execução de captura. Elas ilustram o comportamento no tempo: rampa de usuários virtuais, patamar estável de vazão e latência, CPU constante no teto e erro em 0%. O painel de vazão exibe uma curva por endpoint (a soma delas corresponde à vazão total da Tabela 1).

Figura 4 – Painel do Grafana, cenário A (execução de captura)



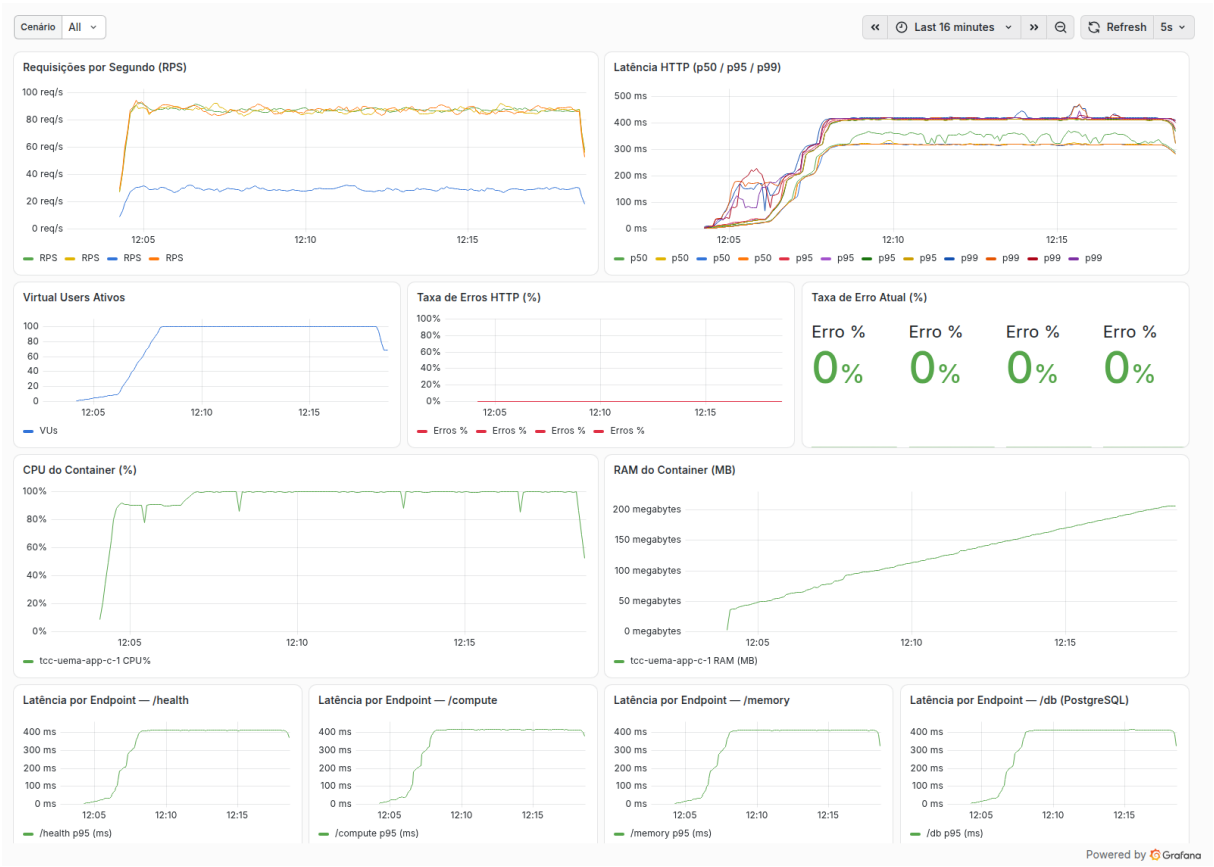
Fonte: elaborado pelo autor (2026).

Figura 5 – Painel do Grafana, cenário B (execução de captura)



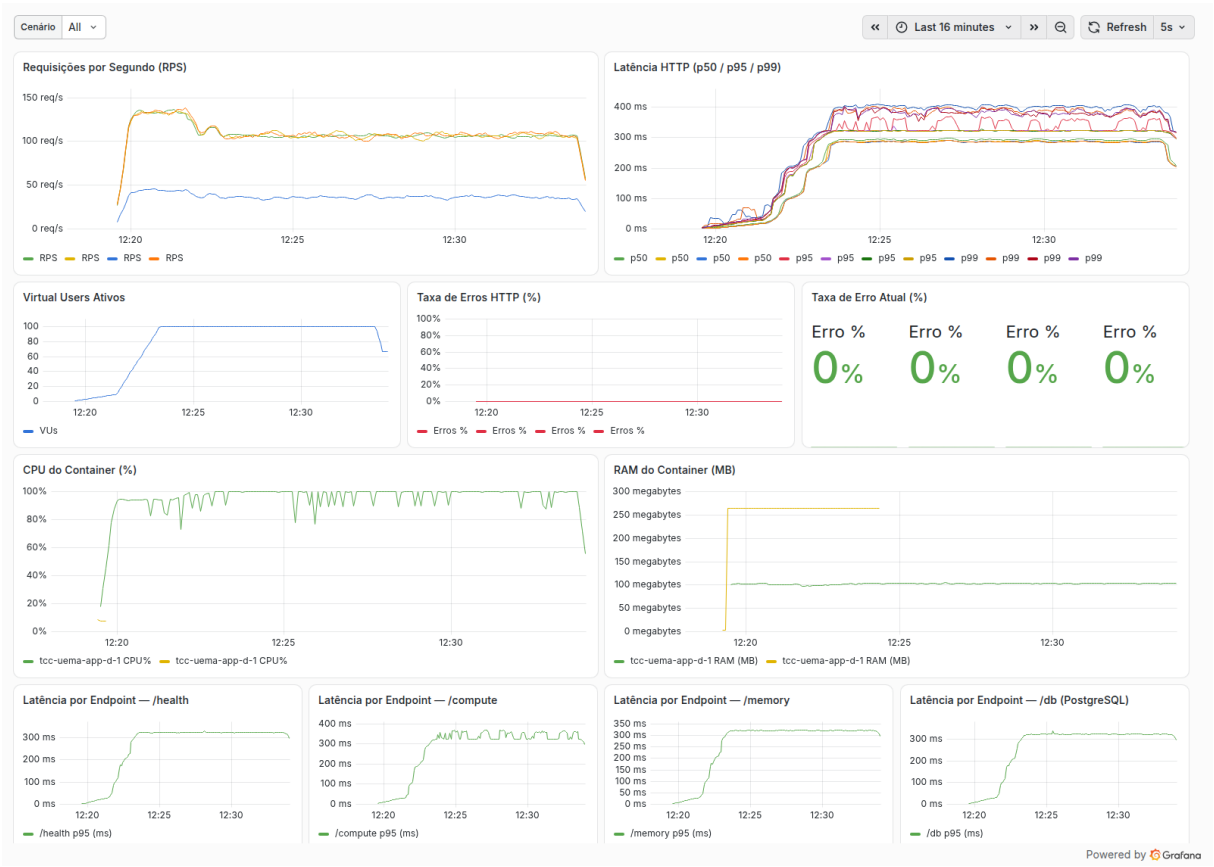
Fonte: elaborado pelo autor (2026).

Figura 6 – Painel do Grafana, cenário C (execução de captura)



Fonte: elaborado pelo autor (2026).

Figura 7 – Painel do Grafana, cenário D (execução de captura)



Nota: a linha amarela curta no início do painel de RAM é uma série de instância anterior do contêiner. Fonte: elaborado pelo autor (2026).

4.6 Discussão

Persistência do processo reduz o custo de inicialização. O resultado mais forte do estudo é que manter o processo e o *framework* carregados em memória mais que dobrou a vazão do CodeIgniter 4 (C em relação a A, +127,7%) com os mesmos recursos, mesmo sem preservar estado de aplicação nem conexão com o banco (Quadro 1). Isso é coerente com a motivação do trabalho: no modelo tradicional, a inicialização do framework se repete a cada requisição, e o modo Worker do FrankenPHP elimina essa repetição ao manter o processo vivo (FRANKENPHP, 2026). Resultado semelhante em direção, ainda que em outro contexto, é relatado por Luz, Silva e Emer (2025), que encontraram vantagem estatisticamente significativa do modelo assíncrono do Swoole sobre o PHP síncrono tradicional em testes de carga, e por Azzahidi, Wijayanto e Darmawan (2025), que incluíram Laravel com FrankenPHP entre os *frameworks* avaliados e o destacaram pelo uso eficiente de memória. Nossos resultados complementam esses estudos ao isolar o efeito do servidor (B) do efeito do modo Worker (C) em um mesmo *framework*.

Trocar apenas o servidor não basta. O FrankenPHP em modo clássico (B) não superou o PHP-FPM (A): ficou 13,4% abaixo. Esse resultado diverge da expectativa inicial, de que o modo clássico, por ser "substituto direto" do PHP-FPM, seria no mínimo equivalente (FRANKENPHP, 2026). Ele reforça a importância do desenho com o cenário B: sem ele, o ganho do cenário C poderia ser atribuído erroneamente ao FrankenPHP em si, quando decorre do modo Worker. Neste experimento não investigamos a causa da diferença entre A e B; hipóteses plausíveis, que exigiriam testes próprios, são o uso de PHP *thread-safe* (necessário ao FrankenPHP) em vez do modelo de processos do PHP-FPM e a contenção entre as *threads* do servidor sob a cota de 1 CPU. Note-se que, no cenário B, a CPU média ficou em 89%, abaixo dos demais, sinal de que parte da cota não foi aproveitada.

O CodeIgniter 4 em Worker ficou atrás do Laravel Octane. O Laravel Octane (D) superou o CodeIgniter 4 em modo Worker (C) em 28% de vazão (368,7 contra 287,7 RPS; C ficou 22% abaixo de D). Esse resultado também diverge da expectativa implícita no problema de pesquisa, de que o CodeIgniter, por ser mais leve, aproveitaria melhor o modo Worker. Três observações ajudam a interpretar: (i) o Octane é um pacote maduro, projetado para esse modelo de execução e já tratando limpeza de estado entre requisições, enquanto o suporte nativo do CodeIgniter ao modo Worker é recente (versão 4.7); (ii) como mostrou o Quadro 1, o Octane reaproveita a conexão com o PostgreSQL e preserva estado entre requisições, ao passo que o CodeIgniter 4 refaz a conexão a cada requisição; como 30% da carga é o endpoint `/db`, isso é uma explicação plausível para parte da diferença, embora a saturação da fila (Seção 4.3) impeça medir esse efeito isoladamente; e (iii) o cenário C apresentou crescimento contínuo de memória, ao passo que o D se estabilizou. Não foi feito perfilamento do código, de modo que essas explicações permanecem hipóteses; o que os dados sustentam é a diferença de vazão e o comportamento da memória, não suas causas.

Quanto a reprodutibilidade e benchmarking. A decisão de repetir cada cenário três vezes, em ordem rotativa, e de reportar o desvio-padrão segue a recomendação da literatura de que benchmarks de software sofrem instabilidade e devem ser tratados com repetição e controle do ambiente (LAABER; BASMACI; SALZA, 2021), e a de que comparações de desempenho exigem condições idênticas de execução (ZHAN, 2022; BRUNST et al., 2022; JIA et al., 2013). O desvio-padrão baixo observado (no máximo 1,8% da média) indica que o ambiente conteinerizado, com limites estritos de CPU e memória, produziu medições estáveis.

4.7 Ameaças à validade e limitações

- **Hospedeiro único e injetor no mesmo hospedeiro:** o k6 rodou na mesma máquina que os servidores (em contêiner com 2,0 CPU), o que pode causar interferência, embora igual entre cenários. Outros processos do computador do autor consumiam memória durante as medições, e o cache do sistema operacional não pôde ser limpo entre execuções por falta de permissão administrativa.
- **Ordem das execuções:** as execuções seguiram ordem rotativa (A, B, C, D) por rodada, com exceção da repetição do cenário D na terceira rodada, refeita isoladamente após falha de rede no *build*.
- **Carga sintética e modelo fechado:** os endpoints são simples e a carga é de usuários virtuais sem tempo de espera, o que satura o servidor e torna a latência função da vazão (Seção 4.3). Os resultados não se estendem automaticamente a aplicações reais, com consultas mais complexas e uso de sessão.
- **Percentil 99 não analisado:** o resumo exportado pelo k6 não trouxe o p99; foram usados p50 e p95.
- **Causas não investigadas:** diferenças entre A e B e entre C e D foram interpretadas como hipóteses, sem perfilamento.

5 CONSIDERAÇÕES FINAIS

Este trabalho investigou se a arquitetura de execução persistente em memória justifica a complexidade de adoção no framework CodeIgniter 4, comparando o PHP-FPM tradicional (cenário A), o FrankenPHP em modo clássico (cenário B) e o FrankenPHP em modo Worker (cenário C), com o Laravel Octane (cenário D) como referência de mercado. O objetivo geral foi medir o impacto dessas arquiteturas na vazão, na latência e no consumo de recursos sob carga, em ambiente conteinerizado com orçamento idêntico de 1,0 CPU e 512 MB por cenário.

Quanto ao primeiro objetivo específico, montar um ambiente controlado e reprodutível, o experimento foi concluído com Docker Compose, limites estritos de recursos, k6 como injetor e Prometheus/Grafana na observação, e produziu medições estáveis (desvio-padrão de até 1,8% da média em três execuções por cenário e 0% de erros). Quanto ao segundo, comparar vazão e latência, os resultados mostram que o modo Worker do FrankenPHP elevou a vazão do CodeIgniter 4 em cerca de 128% em relação ao PHP-FPM (287,7 contra 126,4 RPS) e reduziu a latência mediana de 765 para 308 ms. Quanto ao terceiro, avaliar o consumo de recursos, todos os cenários saturaram a CPU (89% a 101% de um núcleo) e nenhum se aproximou do limite de memória; o custo do modo Worker foi um uso maior de RAM (143 MiB em média contra 49 MiB no PHP-FPM) e, no CodeIgniter 4, um crescimento contínuo de memória durante os 15 minutos de teste, ausente no Laravel Octane.

Respondendo à pergunta de pesquisa, a persistência em memória compensou claramente neste cenário: o ganho de vazão foi superior a duas vezes com os mesmos recursos. Já o FrankenPHP em modo clássico, isoladamente, não entregou ganho (13% abaixo do PHP-FPM), de modo que, para o CodeIgniter 4, a vantagem vem do modo Worker e não da simples troca de servidor. Verificou-se ainda que o modo Worker do CodeIgniter 4 mantém o processo vivo, mas reinicia o estado da aplicação e refaz a conexão com o banco a cada requisição, ao contrário do Laravel Octane, que preserva ambos; assim, o ganho do CodeIgniter decorre da eliminação da inicialização do *framework*. Em relação ao balizador, o Laravel Octane obteve vazão 28% superior ao CodeIgniter 4 em modo Worker, resultado que contraria a expectativa de que a leveza do CodeIgniter lhe daria vantagem e que mostra que a maturidade da integração com o modo Worker pesa tanto quanto o peso do framework.

Como contribuições, o estudo oferece dados empíricos reprodutíveis sobre o suporte nativo do CodeIgniter 4 ao modo Worker (introduzido na versão 4.7), ainda pouco documentado na literatura, identifica o crescimento contínuo de memória nessa configuração e separa o efeito do servidor do efeito do modo de execução. Para equipes que mantêm aplicações

em CodeIgniter 4, os resultados indicam que vale avaliar o modo Worker, ao passo que a migração apenas para o modo clássico do FrankenPHP não se justificaria pelo desempenho.

Entre as limitações, destacam-se: execução em um único hospedeiro, com o injetor de carga na mesma máquina; carga sintética em modelo fechado, que satura o servidor e faz a latência refletir a vazão; ausência de limpeza do cache do sistema operacional; análise limitada a p50 e p95; e causas das diferenças (A contra B e C contra D) tratadas como hipóteses, sem perfilamento do código.

Como trabalhos futuros, sugere-se: investigar o crescimento de memória do CodeIgniter 4 em modo Worker em testes mais longos; repetir o experimento com carga de taxa de chegada fixa (modelo aberto), para isolar o custo de cada endpoint; incluir percentil 99; perfilar o código para explicar a diferença entre o CodeIgniter 4 em Worker e o Laravel Octane; investigar a diferença entre o PHP-FPM e o FrankenPHP clássico; testar aplicações com consultas e sessões mais realistas; e comparar outras alternativas persistentes, como Swoole e RoadRunner.

REFERÊNCIAS

AZZAHIDI, A. S.; WIJAYANTO, B.; DARMAWAN, A. Performance evaluation of backend frameworks for REST API: a comparative study of Spring Boot, Flask, Express.js, Laravel FrankenPHP, and Gin. **Jurnal Teknik Informatika (JUTIF)**, Purwokerto, v. 6, n. 4, p. 2405-2419, ago. 2025. DOI: 10.52436/1.jutif.2025.6.4.4811.

BRUNST, H. et al. First experiences in performance benchmarking with the new SPEChpc 2021 suites. **arXiv**, [S. l.], arXiv:2203.06751v3 [cs.DC], 28 mar. 2022. Disponível em: https://arxiv.org/abs/2203.06751. Acesso em: 15 out. 2026.

CODEIGNITER FOUNDATION. **CodeIgniter 4 user guide**. [S. l.], 2026. Disponível em: https://codeigniter.com/user_guide/. Acesso em: 15 out. 2026.

FRANKENPHP. **FrankenPHP: the modern PHP app server – worker mode**. [S. l.], 2026. Disponível em: https://frankenphp.dev/docs/worker/. Acesso em: 15 out. 2026.

GRAFANA LABS. **Grafana k6 documentation**. [S. l.], 2026. Disponível em: https://grafana.com/docs/k6/latest/. Acesso em: 15 out. 2026.

JIA, Z. et al. The implications of diverse applications and scalable data sets in benchmarking big data systems. **arXiv**, [S. l.], arXiv:1307.7943v1 [cs.PF], 30 jul. 2013. Disponível em: https://arxiv.org/abs/1307.7943. Acesso em: 15 out. 2026.

LAABER, C.; BASMACI, M.; SALZA, P. Predicting unstable software benchmarks using static source code features. **Empirical Software Engineering**, [S. l.], v. 26, art. 114, 2021. DOI: 10.1007/s10664-021-09996-y.

LARAVEL. **Laravel Octane**. [S. l.], 2026. Disponível em: https://laravel.com/docs/octane. Acesso em: 15 out. 2026.

LUZ, J. M. da; SILVA, R. S. da; EMER, C. D. **Otimização de aplicações web com programação assíncrona em PHP e Swoole**. Veranópolis: Instituto Federal do Rio Grande do Sul, 2025. [CONFERIR: ano e evento de publicação.]

ZHAN, J. A BenchCouncil view on benchmarking emerging and future computing. **arXiv**, [S. l.], arXiv:2205.07769v1 [cs.ET], 16 maio 2022. Disponível em: https://arxiv.org/abs/2205.07769. Acesso em: 15 out. 2026.

APÊNDICE A — ORQUESTRAÇÃO DOS CONTÊINERES (DOCKER-COMPOSE.YML)

docker-compose.yml

```
services:

  # ==========================================================================
  # CENÁRIO A — Legacy: CodeIgniter 4 + PHP-FPM (Alpine) + Nginx
  # Topologia real de produção PHP tradicional
  # ==========================================================================
  app-a:
    profiles: [scenario-a]
    build:
      context: .
      dockerfile: docker/scenario-a/Dockerfile
    expose:
      - "9000"
    deploy:
      resources:
        limits:
          # 0.85 CPU / 448MB — orçamento do processo PHP-FPM
          # nginx-a recebe 0.15 CPU / 64MB: juntos totalizam 1.0 CPU / 512MB
          # Isso iguala o orçamento total do Cenário A aos demais (B, C, D)
          cpus: "0.85"
          memory: "448M"
    environment:
      SCENARIO_NAME: scenario-a
      COMPUTE_SIZE: "1000"
      CI_ENVIRONMENT: production
    volumes:
      - ./docker/scenario-a/php-fpm.conf:/usr/local/etc/php-fpm.d/www.conf
    depends_on:
      postgres:
        condition: service_healthy
    networks:
      - bench-net
    restart: unless-stopped

  nginx-a:
    profiles: [scenario-a]
    image: nginx:alpine
    ports:
      - "8080:8080"
    deploy:
      resources:
        limits:
          # 0.15 CPU / 64MB — orçamento do reverse proxy Nginx
          # Nginx é muito leve; esse limite reflete seu papel no stack
          # Total Cenário A: app-a (0.85/448M) + nginx-a (0.15/64M) = 1.0 CPU / 512MB
          cpus: "0.15"
          memory: "64M"
    volumes:
      - ./docker/scenario-a/nginx.conf:/etc/nginx/conf.d/default.conf
      - ./apps/ci4-base/public:/var/www/html/public:ro
    depends_on:
      - app-a
    networks:
      - bench-net
    restart: unless-stopped

  # ==========================================================================
  # CENÁRIO B — Modern Classic: CodeIgniter 4 + FrankenPHP Classic Mode
  # FrankenPHP como substituto do PHP-FPM (sem Worker, sem estado persistente)
  # Isola o ganho do motor FrankenPHP vs o Worker Mode em si
  # ==========================================================================
  app-b:
    profiles: [scenario-b]
    build:
      context: .
      dockerfile: docker/scenario-b/Dockerfile
    ports:
```

```
    - "8080:8080"
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: "512M"
  environment:
    SCENARIO_NAME: scenario-b
    COMPUTE_SIZE: "1000"
    CI_ENVIRONMENT: production
    SERVER_NAME: ":8080"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - bench-net
  restart: unless-stopped

# ============================================================================
# CENÁRIO C — Vanguard: CodeIgniter 4 + FrankenPHP Worker Mode
# Bridge customizada mantém o CI4 em memória persistente entre requests
# Este é o cenário central do TCC
# ============================================================================
app-c:
  profiles: [scenario-c]
  build:
    context: .
    dockerfile: docker/scenario-c/Dockerfile
  ports:
    - "8080:8080"
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: "512M"
  environment:
    SCENARIO_NAME: scenario-c
    COMPUTE_SIZE: "1000"
    CI_ENVIRONMENT: production
    SERVER_NAME: ":8080"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - bench-net
  restart: unless-stopped

# ============================================================================
# CENÁRIO D — Referência: Laravel + Laravel Octane + FrankenPHP
# Benchmark de mercado — Octane suporta Worker Mode nativamente
# ============================================================================
app-d:
  profiles: [scenario-d]
  build:
    context: .
    dockerfile: docker/scenario-d/Dockerfile
  ports:
    - "8080:8080"
  deploy:
    resources:
      limits:
        cpus: "1.0"
        memory: "512M"
  environment:
    SCENARIO_NAME: scenario-d
    COMPUTE_SIZE: "1000"
    APP_ENV: production
    APP_DEBUG: "false"
    OCTANE_SERVER: frankenphp
    SERVER_NAME: ":8080"
  depends_on:
    postgres:
      condition: service_healthy
  networks:
    - bench-net
  restart: unless-stopped
```

```
# ==============================================================================
# BANCO DE DADOS — PostgreSQL 16
# Compartilhado entre todos os cenários (roda um de cada vez via profiles)
# Recurso isolado: NÃO conta no orçamento de 1.0 CPU / 512MB dos apps
#
# Por que PostgreSQL para o benchmark:
#   - Modelo process-per-connection (~5-10MB RAM/conexão) torna o custo de
#     abrir uma nova conexão mais caro que MySQL (thread-based)
#   - Amplifica exatamente o diferencial do Worker Mode (conexão persistente)
#   - Cenário A (PHP-FPM): nova conexão PG por request = overhead mensurável
#   - Cenário C (Worker Mode): conexão reutilizada = sem overhead de handshake
# ==============================================================================
postgres:
  profiles: [scenario-a, scenario-b, scenario-c, scenario-d]
  image: postgres:16-alpine
  environment:
    POSTGRES_DB:       benchmark
    POSTGRES_USER:     benchmark
    POSTGRES_PASSWORD: benchmark
  command: >
    postgres
      -c max_connections=20
      -c shared_buffers=64MB
      -c log_min_messages=WARNING
  volumes:
    - ./docker/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    - postgres-data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U benchmark -d benchmark"]
    interval: 5s
    timeout: 3s
    retries: 10
    start_period: 10s
  deploy:
    resources:
      limits:
        cpus: "0.5"
        memory: "256M"
  networks:
    - bench-net
  restart: unless-stopped


# ==============================================================================
# INJETOR DE CARGA — Grafana k6
# 2.0 CPU / 512MB RAM conforme especificado no TCC
# Executa apenas via `docker compose run --rm k6 run /scripts/...`
# ==============================================================================
k6:
  profiles: [scenario-a, scenario-b, scenario-c, scenario-d]
  image: grafana/k6:latest
  user: root
  volumes:
    - ./k6:/scripts
    - ./results:/results
  environment:
    K6_PROMETHEUS_RW_SERVER_URL: http://prometheus:9090/api/v1/write
    K6_PROMETHEUS_RW_TREND_AS_NATIVE_HISTOGRAM: "true"
  deploy:
    resources:
      limits:
        cpus: "2.0"
        memory: "512M"
  networks:
    - bench-net
  depends_on:
    - prometheus


# ==============================================================================
# OBSERVABILIDADE
# ==============================================================================


prometheus:
  profiles: [scenario-a, scenario-b, scenario-c, scenario-d]
  image: prom/prometheus:latest
```

```
    ports:
      - "9090:9090"
    volumes:
      - ./monitoring/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml
      - prometheus-data:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.path=/prometheus"
      - "--web.enable-remote-write-receiver"
      - "--storage.tsdb.retention.time=7d"
    networks:
      - bench-net
    restart: unless-stopped


  grafana:
    profiles: [scenario-a, scenario-b, scenario-c, scenario-d]
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: benchmark
      GF_AUTH_ANONYMOUS_ENABLED: "true"
      GF_AUTH_ANONYMOUS_ORG_ROLE: Viewer
      GF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH: /var/lib/grafana/dashboards/benchmark.json
    volumes:
      - ./monitoring/grafana/provisioning:/etc/grafana/provisioning
      - ./monitoring/grafana/dashboards:/var/lib/grafana/dashboards
      - grafana-data:/var/lib/grafana
    networks:
      - bench-net
    depends_on:
      - prometheus
    restart: unless-stopped


  # cAdvisor — expõe CPU/RAM dos containers em formato Prometheus
  cadvisor:
    profiles: [scenario-a, scenario-b, scenario-c, scenario-d]
    image: gcr.io/cadvisor/cadvisor:latest
    ports:
      - "8081:8080"
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker/:/var/lib/docker:ro
    networks:
      - bench-net
    restart: unless-stopped

networks:
  bench-net:
    driver: bridge

volumes:
  prometheus-data:
  grafana-data:
  postgres-data:
```

APÊNDICE B — CONFIGURAÇÃO DOS SERVIDORES E IMAGENS DOS CENÁRIOS

docker/scenario-a/Dockerfile

```
FROM php:8.3-fpm-alpine

# icu-dev: necessário para ext-intl (requerida pelo CI4)
# libpq-dev: necessário para ext-pgsql/pdo_pgsql
RUN apk add --no-cache icu-dev libpq-dev git unzip \
    && docker-php-ext-install opcache intl pgsql pdo_pgsql

# OPcache para benchmark justo (Scenario A recebe todos os otimizadores que tem direito)
COPY docker/scenario-a/opcache.ini /usr/local/etc/php/conf.d/opcache.ini

# Instalar composer
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /var/www/html

# Copiar código da aplicação
COPY apps/ci4-base/ .

# Instalar dependências
RUN composer install --no-dev --optimize-autoloader --no-interaction --no-scripts

# Permissões de escrita para o CI4
RUN mkdir -p writable/{cache,logs,session,uploads,debugbar} \
    && chmod -R 777 writable/

# Variáveis de ambiente sobrescritas pelo docker-compose
ENV SCENARIO_NAME=scenario-a
ENV COMPUTE_SIZE=1000
ENV CI_ENVIRONMENT=production

EXPOSE 9000
CMD ["php-fpm"]
```

docker/scenario-a/php-fpm.conf

```
[www]
user  = www-data
group = www-data

listen = 0.0.0.0:9000

; Pool estático com 4 workers — mesma contagem dos Workers nos Cenários B/C/D
; Variável controlada nos 4 cenários para comparação justa
pm = static
pm.max_children = 4

; Desabilitar logs de acesso do FPM (reduz I/O nas medições)
access.log = /dev/null
```

docker/scenario-a/nginx.conf

```
server {
    listen 8080;
    root /var/www/html/public;
    index index.php;

    # Desabilita access log para eliminar I/O de disco das medições
    access_log off;
    error_log /var/log/nginx/error.log warn;
```

```
    location / {
        try_files $uri $uri/ /index.php?$query_string;
    }

    location ~ \.php$ {
        fastcgi_pass          app-a:9000;
        fastcgi_index         index.php;
        fastcgi_param         SCRIPT_FILENAME $document_root$fastcgi_script_name;
        fastcgi_read_timeout  60s;
        include               fastcgi_params;
    }

    location ~ /\. {
        deny all;
    }
}
```

docker/scenario-b/Dockerfile

```
FROM dunglas/frankenphp:latest-php8.3

# git necessário para o Composer em imagens Debian
RUN apt-get update && apt-get install -y --no-install-recommends git unzip && rm -rf
/var/lib/apt/lists/*

# Extensões para CI4
RUN install-php-extensions opcache intl pgsql pdo_pgsql

# OPcache — mesmo nível de otimização do Cenário A para comparação justa
COPY docker/scenario-b/opcache.ini /usr/local/etc/php/conf.d/opcache.ini

# Instalar composer
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /app

# Copiar código da aplicação CI4
COPY apps/ci4-base/ .

# Instalar dependências
RUN composer install --no-dev --optimize-autoloader --no-interaction --no-scripts

# Permissões de escrita para o CI4
RUN mkdir -p writable/{cache,logs,session,uploads,debugbar} \
    && chmod -R 777 writable/

# Caddyfile para modo Classic (sem diretiva worker — comportamento igual ao PHP-FPM)
COPY docker/scenario-b/Caddyfile /etc/caddy/Caddyfile

ENV SCENARIO_NAME=scenario-b
ENV COMPUTE_SIZE=1000
ENV CI_ENVIRONMENT=production
ENV SERVER_NAME=":8080"

EXPOSE 8080
```

docker/scenario-b/Caddyfile

```
{
    # FrankenPHP modo Classic — sem diretiva worker
    # Cada request inicia um novo contexto PHP (equivalente ao PHP-FPM)
    frankenphp
    order php_server before file_server
    admin off
}

:8080 {
    root * /app/public
```

```
    php_server

    log {
        output discard
    }
}
```

docker/scenario-c/Dockerfile

```
FROM dunglas/frankenphp:latest-php8.3

# git necessário para o Composer em imagens Debian
RUN apt-get update && apt-get install -y --no-install-recommends git unzip && rm -rf
/var/lib/apt/lists/*

# Extensões para CI4
RUN install-php-extensions opcache intl pgsql pdo_pgsql

# OPcache — mesmo nível dos outros cenários
COPY docker/scenario-c/opcache.ini /usr/local/etc/php/conf.d/opcache.ini

# Instalar composer
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /app

# Copiar código da aplicação CI4
# Inclui public/frankenphp-worker.php (suporte nativo CI4 >= 4.7.0)
COPY apps/ci4-base/ .

# Instalar dependências (CI4 >= 4.7 para suporte nativo ao Worker Mode)
RUN composer install --no-dev --optimize-autoloader --no-interaction --no-scripts

# Gera public/frankenphp-worker.php via comando oficial do CI4
RUN php spark worker:install --no-interaction

# Permissões de escrita para o CI4
RUN mkdir -p writable/{cache,logs,session,uploads,debugbar} \
    && chmod -R 777 writable/

# Caddyfile oficial gerado pelo `php spark worker:install`
# Worker mode com 4 workers (mesmo número de pm.max_children do Cenário A)
COPY docker/scenario-c/Caddyfile /etc/caddy/Caddyfile

ENV SCENARIO_NAME=scenario-c
ENV COMPUTE_SIZE=1000
ENV CI_ENVIRONMENT=production
ENV SERVER_NAME=":8080"

EXPOSE 8080
```

docker/scenario-c/Caddyfile

```
{
    frankenphp {
        worker {
            file public/frankenphp-worker.php
            num 4
        }
    }
    order php_server before file_server
    admin off
}

:8080 {
    root * public
```

```
    php_server {
        try_files {path} frankenphp-worker.php
    }

    file_server

    log {
        output discard
    }
}
```

docker/scenario-d/Dockerfile

```
FROM dunglas/frankenphp:latest-php8.3

# git necessário para o Composer em imagens Debian
RUN apt-get update && apt-get install -y --no-install-recommends git unzip && rm -rf
/var/lib/apt/lists/*

# Extensões para Laravel
RUN install-php-extensions opcache pcntl pgsql pdo_pgsql

# OPcache — mesmo nível dos outros cenários
COPY docker/scenario-d/opcache.ini /usr/local/etc/php/conf.d/opcache.ini

# Instalar composer
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer

WORKDIR /app

# Copiar app Laravel
COPY apps/laravel-octane/ .

# Instalar dependências
RUN composer update --no-dev --optimize-autoloader --no-interaction --no-scripts

# Otimizações de produção do Laravel
RUN php artisan config:cache \
    && php artisan route:cache \
    && php artisan event:cache

# Permissões de armazenamento
RUN chmod -R 777 storage bootstrap/cache

ENV SCENARIO_NAME=scenario-d
ENV COMPUTE_SIZE=1000
ENV APP_ENV=production
ENV APP_DEBUG=false
ENV OCTANE_SERVER=frankenphp
ENV SERVER_NAME=":8080"

EXPOSE 8080

# Octane com FrankenPHP — 4 workers para comparação justa com os outros cenários
CMD ["php", "artisan", "octane:frankenphp", "--workers=4", "--port=8080", "--host=0.0.0.0"]
```

docker/scenario-c/opcache.ini

```
opcache.enable=1
opcache.memory_consumption=128
opcache.interned_strings_buffer=8
opcache.max_accelerated_files=10000
opcache.validate_timestamps=0
opcache.revalidate_freq=0
opcache.save_comments=0
opcache.fast_shutdown=1
```

APÊNDICE C — SCRIPTS DE CARGA DO K6

k6/load.js

```
/**
 * Load Test — Teste de carga principal do TCC
 *
 * Distribuição de endpoints:
 *   10% health  — latência base
 *   30% compute — CPU-bound puro (sem I/O)
 *   30% memory  — persistência de estado em memória
 *   30% db      — persistência de conexão PostgreSQL (principal diferencial Worker Mode)
 *
 * Métricas via Prometheus remote_write → Grafana: http://localhost:3000
 *
 * Execução:
 *   docker compose --profile scenario-X run --rm k6 \
 *     run /scripts/load.js -e TARGET_URL=http://app-X:8080
 */

import { testHealth, testCompute, testMemory, testDb } from './lib/endpoints.js';

export const options = {
  stages: [
    { duration: '2m',  target: 10  },  // aquecimento
    { duration: '2m',  target: 100 },  // rampa de carga
    { duration: '10m', target: 100 },  // pico sustentado
    { duration: '1m',  target: 0   },  // desaceleração
  ],

  // Tag global: todas as métricas enviadas ao Prometheus carregam o label `scenario`.
  // Permite filtrar por cenário no Grafana mesmo com o TSDB acumulando múltiplos runs.
  tags: {
    scenario: __ENV.SCENARIO_NAME || 'unknown',
  },

  thresholds: {
    http_req_failed:             ['rate<0.05'],
    http_req_duration:           ['p(99)<5000'],
    'endpoint_health_duration':  ['p(95)<5000'],
    'endpoint_compute_duration': ['p(95)<5000'],
    'endpoint_memory_duration':  ['p(95)<5000'],
    'endpoint_db_duration':      ['p(95)<5000'],
  },
};

export default function () {
  const rand = Math.random();

  if (rand < 0.10) {
    testHealth();
  } else if (rand < 0.40) {
    testCompute();
  } else if (rand < 0.70) {
    testMemory();
  } else {
    testDb();
  }
}
```

k6/lib/endpoints.js

```
import http from 'k6/http';
import { check } from 'k6';
import { Trend } from 'k6/metrics';

export const healthDuration  = new Trend('endpoint_health_duration',  true);
export const computeDuration = new Trend('endpoint_compute_duration', true);
export const memoryDuration  = new Trend('endpoint_memory_duration',  true);
export const dbDuration      = new Trend('endpoint_db_duration',      true);
```

```
const BASE_URL = __ENV.TARGET_URL || 'http://localhost:8080';

export function testHealth() {
  const res = http.get(`${BASE_URL}/health`, { tags: { endpoint: 'health' } });
  healthDuration.add(res.timings.duration);
  check(res, {
    'health: status 200':         (r) => r.status === 200,
    'health: body has status ok': (r) => r.json('status') === 'ok',
  });
  return res;
}

export function testCompute() {
  const res = http.get(`${BASE_URL}/compute`, { tags: { endpoint: 'compute' } });
  computeDuration.add(res.timings.duration);
  check(res, {
    'compute: status 200':          (r) => r.status === 200,
    'compute: body has data array': (r) => Array.isArray(r.json('data')),
  });
  return res;
}

export function testMemory() {
  const res = http.get(`${BASE_URL}/memory`, { tags: { endpoint: 'memory' } });
  memoryDuration.add(res.timings.duration);
  check(res, {
    'memory: status 200':     (r) => r.status === 200,
    'memory: counter exists': (r) => r.json('counter') > 0,
    'memory: pid exists':     (r) => r.json('pid') > 0,
  });
  return res;
}

export function testDb() {
  const res = http.get(`${BASE_URL}/db`, { tags: { endpoint: 'db' } });
  dbDuration.add(res.timings.duration);
  check(res, {
    'db: status 200':      (r) => r.status === 200,
    'db: item returned':   (r) => r.json('item') !== null,
    'db: item has id':     (r) => r.json('item.id') > 0,
  });
  return res;
}
```

k6/smoke.js

```
/**
 * Smoke Test — Sanidade do ambiente
 *
 * Execução:
 *   docker compose --profile scenario-X run --rm k6 \
 *     run /scripts/smoke.js -e TARGET_URL=http://app-X:8080
 */

import { sleep } from 'k6';
import { testHealth, testCompute, testMemory, testDb } from './lib/endpoints.js';

export const options = {
  vus:      1,
  duration: '1m',

  // Tag global: todas as métricas enviadas ao Prometheus carregam o label `scenario`.
  // Permite filtrar por cenário no Grafana mesmo com o TSDB acumulando múltiplos runs.
  tags: {
    scenario: __ENV.SCENARIO_NAME || 'unknown',
  },

  thresholds: {
    http_req_failed:             ['rate<0.01'],
    http_req_duration:           ['p(95)<500'],
```

```
    'endpoint_health_duration':  ['p(95)<100'],
    'endpoint_compute_duration': ['p(95)<1000'],
    'endpoint_memory_duration':  ['p(95)<200'],
    'endpoint_db_duration':      ['p(95)<300'],
  },
};

export default function () {
  testHealth();
  sleep(0.1);

  testCompute();
  sleep(0.1);

  testMemory();
  sleep(0.1);

  testDb();
  sleep(0.1);
}
```

APÊNDICE D — SCRIPTS DE AUTOMAÇÃO DOS TESTES

scripts/run-scenario.sh

```
#!/usr/bin/env bash
# ================================================================================
# run-scenario.sh — Executa um cenário de benchmark de forma isolada e limpa
#
# Uso:
#   ./scripts/run-scenario.sh <cenário> <teste> [opções]
#
# Exemplos:
#   ./scripts/run-scenario.sh a smoke          # Cenário A, smoke test
#   ./scripts/run-scenario.sh b load           # Cenário B, load test
#   ./scripts/run-scenario.sh c load --no-cache-clear  # sem limpeza de cache
#
# Cenários disponíveis: a | b | c | d
# Testes disponíveis:   smoke | load
# ================================================================================

set -euo pipefail

SCENARIO="${1:-}"
TEST="${2:-smoke}"
NO_CACHE_CLEAR=false

for arg in "${@:3}"; do
  [[ "$arg" == "--no-cache-clear" ]] && NO_CACHE_CLEAR=true
done

# Validações
if [[ -z "$SCENARIO" ]]; then
  echo "Erro: informe o cenário. Uso: $0 <a|b|c|d> <smoke|load>"
  exit 1
fi

if [[ ! "$SCENARIO" =~ ^[abcd]$ ]]; then
  echo "Erro: cenário inválido '$SCENARIO'. Use: a, b, c, ou d"
  exit 1
fi

if [[ ! "$TEST" =~ ^(smoke|load)$ ]]; then
  echo "Erro: teste inválido '$TEST'. Use: smoke ou load"
  exit 1
fi

PROFILE="scenario-${SCENARIO}"

# Determina o host alvo do k6 (Cenário A usa nginx-a como proxy, demais usam app-X
diretamente)
if [[ "$SCENARIO" == "a" ]]; then
  TARGET_HOST="nginx-a"
else
  TARGET_HOST="app-${SCENARIO}"
fi
TARGET_URL="http://${TARGET_HOST}:8080"

echo "================================================================================"
echo " TCC UEMA — Benchmark PHP Architecture"
echo " Cenário:  ${PROFILE^^} (${TARGET_HOST})"
echo " Teste:    ${TEST}"
echo " Data/hora: $(date '+%Y-%m-%d %H:%M:%S')"
echo "================================================================================"

# --- Passo 1: Derrubar qualquer cenário em execução e limpar métricas anteriores ---
echo ""
echo "[1/6] Derrubando cenários anteriores..."
docker compose \
  --profile scenario-a \
  --profile scenario-b \
```

```
  --profile scenario-c \
  --profile scenario-d \
  down --remove-orphans 2>/dev/null || true

# Remove o volume do Prometheus para garantir que cada cenário comece com
# TSDB limpo. Sem isso, métricas de cenários anteriores ficam no banco e
# podem confundir a análise — especialmente em queries PromQL sem filtro de tempo.
# O grafana-data e postgres-data são preservados intencionalmente:
#   - grafana-data: dashboards e datasources provisionados não precisam resetar
#   - postgres-data: tabela read-only, dados determinísticos, init.sql já rodou
PROMETHEUS_VOLUME="$(basename "$(pwd)")_prometheus-data"
if docker volume inspect "$PROMETHEUS_VOLUME" &>/dev/null; then
  docker volume rm "$PROMETHEUS_VOLUME" > /dev/null
  echo "   Volume do Prometheus limpo (${PROMETHEUS_VOLUME})."
fi

# --- Passo 2: Limpar cache do sistema operacional ---
# Crítico para isolar os benchmarks: evita que resultados do cenário anterior
# influenciem o próximo por páginas de memória, dentries e inodes cacheados.
if [[ "$NO_CACHE_CLEAR" == false ]]; then
  echo ""
  echo "[2/6] Limpando cache do sistema operacional (page cache, dentries, inodes)..."

  if [[ "$(uname)" == "Linux" ]]; then
    if [[ -w /proc/sys/vm/drop_caches ]]; then
      # Sincroniza buffers pendentes antes de liberar o cache
      sync
      # 3 = libera page cache + dentries + inodes
      echo 3 | sudo tee /proc/sys/vm/drop_caches > /dev/null
      echo "   Cache do SO liberado com sucesso."
    else
      echo "   AVISO: sem permissão para limpar o cache do SO."
      echo "   Execute: echo 3 | sudo tee /proc/sys/vm/drop_caches"
      echo "   Ou adicione ao sudoers: $(whoami) ALL=(ALL) NOPASSWD: /usr/bin/tee
/proc/sys/vm/drop_caches"
      echo "   Continuando sem limpeza de cache..."
    fi
  elif [[ "$(uname)" == "Darwin" ]]; then
    sudo purge 2>/dev/null && echo "   Cache do macOS liberado (purge)." || \
      echo "   AVISO: falha ao executar purge. Continuando..."
  else
    echo "   AVISO: sistema não reconhecido para limpeza de cache."
  fi
else
  echo ""
  echo "[2/6] Limpeza de cache do SO ignorada (--no-cache-clear)."
fi

# --- Passo 3: Aguardar memória assentar ---
echo ""
echo "[3/6] Aguardando 3s para memória assentar..."
sleep 3

# --- Passo 4: Subir o cenário + stack de observabilidade ---
echo ""
echo "[4/6] Iniciando ${PROFILE} + Prometheus + Grafana + cAdvisor..."
docker compose --profile "$PROFILE" up -d --build

# --- Passo 5: Aguardar o app estar pronto ---
echo ""
echo "[5/6] Aguardando ${TARGET_HOST} responder em /health..."
MAX_ATTEMPTS=30
ATTEMPT=0
until docker compose --profile "$PROFILE" run --rm --no-deps k6 \
  run --quiet \
  --duration 1s --vus 1 \
  -e TARGET_URL="$TARGET_URL" \
  -e SCRIPT=smoke \
  /scripts/smoke.js &>/dev/null; do
  ATTEMPT=$((ATTEMPT + 1))
  if [[ $ATTEMPT -ge $MAX_ATTEMPTS ]]; then
    echo "   ERRO: ${TARGET_HOST} não respondeu após ${MAX_ATTEMPTS} tentativas."
    echo "   Verifique: docker compose --profile $PROFILE logs"
    exit 1
```

```
  fi
  echo "   Tentativa ${ATTEMPT}/${MAX_ATTEMPTS}... aguardando 2s"
  sleep 2
done
echo "   ${TARGET_HOST} está pronto!"

# --- Passo 6: Executar o teste k6 ---
echo ""
echo "[6/6] Executando ${TEST} test contra ${TARGET_URL}..."
echo "      Grafana disponível em http://localhost:3000 (admin / benchmark)"
echo ""

RESULTS_DIR="$(dirname "$0")/../results"
mkdir -p "$RESULTS_DIR"
TIMESTAMP="$(date '+%Y%m%d_%H%M%S')"
RESULTS_TXT="/results/scenario-${SCENARIO}-${TEST}-${TIMESTAMP}.txt"
DASHBOARD_HTML="/results/dashboard-${PROFILE}-${TEST}-${TIMESTAMP}.html"

docker compose --profile "$PROFILE" run --rm \
  -e K6_WEB_DASHBOARD=true \
  -e K6_WEB_DASHBOARD_EXPORT="$DASHBOARD_HTML" \
  k6 run \
  --out experimental-prometheus-rw \
  --summary-export /results/last-run.json \
  -e TARGET_URL="$TARGET_URL" \
  -e SCENARIO_NAME="$PROFILE" \
  "/scripts/${TEST}.js" \
  | tee "${RESULTS_DIR}/scenario-${SCENARIO}-${TEST}-${TIMESTAMP}.txt"

echo ""
echo "========================================================================"
echo " Teste concluído!"
echo " Resultados: ${RESULTS_DIR}/scenario-${SCENARIO}-${TEST}-${TIMESTAMP}.txt
 Dashboard:  ${RESULTS_DIR}/dashboard-${PROFILE}-${TEST}-${TIMESTAMP}.html"
echo " Grafana:    http://localhost:3000"
echo " Prometheus: http://localhost:9090"
echo "========================================================================"
echo ""
echo " Para coletar snapshot de docker stats execute:"
echo "   ./scripts/collect-results.sh ${SCENARIO}"
echo ""
echo " Para derrubar o ambiente execute:"
echo "   docker compose --profile ${PROFILE} down"
echo "========================================================================"
```

scripts/run-matrix.sh

```
#!/usr/bin/env bash
# run-matrix.sh — Executa N rodadas de load test em ordem rotativa (a,b,c,d) e
# arquiva cada rodada em results/runs/<cenário>-run<N>/ (summary k6, dashboard, log, docker
stats, env).
# Uso: ./scripts/run-matrix.sh [rodadas=3]
set -uo pipefail
cd "$(dirname "$0")/.."
ROUNDS="${1:-3}"
mkdir -p results/runs
for r in $(seq 1 "$ROUNDS"); do
  for s in a b c d; do
    dir="results/runs/${s}-run${r}"
    [[ -f "$dir/last-run.json" ]] && { echo "pulando $dir (já existe)"; continue; }
    mkdir -p "$dir"
    rm -f results/last-run.json
    ./scripts/run-scenario.sh "$s" load --no-cache-clear 2>&1 | tee "$dir/console.log"
    docker stats --no-stream --format '{{.Name}},{{.CPUPerc}},{{.MemUsage}}' > "$dir/docker-
stats-post.csv" 2>/dev/null || true
    mv results/last-run.json "$dir/" 2>/dev/null
    mv results/dashboard-scenario-${s}-load-*.html "$dir/" 2>/dev/null
    mv results/scenario-${s}-load-*.txt "$dir/" 2>/dev/null
    date -Is > "$dir/finished_at"
  done
done
docker compose --profile scenario-a --profile scenario-b --profile scenario-c --profile
scenario-d down --remove-orphans
```

scripts/run-captura.sh

```
#!/usr/bin/env bash
# Rodada extra de captura (Grafana tema claro, Chrome headless). Não entra nas médias
oficiais.
# Tira o print do dashboard no minuto ~14:20 de cada rodada (janela de 16 min mostra a rodada
inteira).
cd "$(dirname "$0")/.."
URL='http://localhost:3000/d/tcc-uema-benchmark?kiosk&theme=light&from=now-16m&to=now'
for s in a b c d; do
  log=results/captura/console-$s.log
  ( until grep -q "running (14m2" "$log" 2>/dev/null; do sleep 2; done
    timeout 90 google-chrome --headless=new --no-sandbox --disable-gpu
--user-data-dir=/tmp/claude-1000/chrome-prof \
      --window-size=1600,1140 --virtual-time-budget=20000 --screenshot="docs/figuras/grafana-
$s.png" "$URL" >/dev/null 2>&1 ) &
  ./scripts/run-scenario.sh "$s" load --no-cache-clear > "$log" 2>&1
  wait
  mkdir -p results/captura/$s
  mv results/dashboard-scenario-$s-load-*.html results/scenario-$s-load-*.txt results/last-
run.json results/captura/$s/ 2>/dev/null
done
docker compose --profile scenario-a --profile scenario-b --profile scenario-c --profile
scenario-d down --remove-orphans
```

scripts/sample-stats.sh

```
#!/usr/bin/env bash
# sample-stats.sh — amostra docker stats a cada 5 s (timestamp,container,cpu%,mem) durante a
matriz
OUT="${1:-results/runs/stats-samples.csv}"
echo "ts,name,cpu_pct,mem_usage" > "$OUT"
while pgrep -f run-matrix.sh >/dev/null; do
  ts=$(date +%s)
  docker stats --no-stream --format '{{.Name}},{{.CPUPerc}},{{.MemUsage}}' 2>/dev/null | grep
-E 'app-[abcd]|nginx-a' | sed "s/^/$ts,/" >> "$OUT"
  sleep 5
done
```

scripts/analyze.py

```
#!/usr/bin/env python3
"""Agrega as rodadas em results/runs/<cenário>-run<N>/last-run.json (k6 --summary-export)
e gera tabelas (Markdown/CSV) e figuras (PNG) em results/analysis/.

Uso: python3 scripts/analyze.py   (requer matplotlib para as figuras)
"""
import csv
import re
import json
import statistics as st
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
RUNS = ROOT / "results" / "runs"
OUT = ROOT / "results" / "analysis"
OUT.mkdir(parents=True, exist_ok=True)

NOMES = {
    "a": "A — CI4 + PHP-FPM",
    "b": "B — CI4 + FrankenPHP clássico",
    "c": "C — CI4 + FrankenPHP Worker",
    "d": "D — Laravel Octane",
}
ENDPOINTS = ["health", "compute", "memory", "db"]
CORES = {"a": "#8c8c8c", "b": "#4c78a8", "c": "#e07b39", "d": "#59a14f"}



def load():
    data = {s: [] for s in NOMES}
    for p in sorted(RUNS.glob("*-run*/last-run.json")):
```

```
        s = p.parent.name.split("-")[0]
        m = json.load(open(p))["metrics"]
        run = {
            "rps": m["http_reqs"]["rate"],
            "reqs": m["http_reqs"]["count"],
            "err": m["http_req_failed"].get("value", m["http_req_failed"].get("rate", 0)),
            "avg": m["http_req_duration"]["avg"],
            "p50": m["http_req_duration"]["med"],
            "p95": m["http_req_duration"]["p(95)"],
            "p99": m["http_req_duration"].get("p(99)", float("nan")),
        }
        for e in ENDPOINTS:
            k = f"endpoint_{e}_duration"
            if k in m:
                run[f"{e}_avg"] = m[k]["avg"]
                run[f"{e}_p95"] = m[k]["p(95)"]
        data[s].append(run)
    return data



def recursos():
    """CPU/RAM por cenário a partir de results/runs/stats-samples.csv (docker stats a cada ~7
s).
    Janela do patamar sustentado de cada rodada: [fim-11,5 min, fim-1,5 min]. Cenário A soma
app-a + nginx-a."""
    from datetime import datetime
    amostras = list(csv.DictReader(open(RUNS / "stats-samples.csv")))
    res = {s: [] for s in NOMES}
    for fin in sorted(RUNS.glob("*-run*/finished_at")):
        s = fin.parent.name.split("-")[0]
        if not (fin.parent / "last-run.json").exists():
            continue
        t1 = datetime.fromisoformat(fin.read_text().strip()).timestamp() - 90
        t0 = t1 - 600
        por_ts = {}
        for r in amostras:
            ts = int(r["ts"])
            if not (t0 <= ts <= t1) or not r["name"].startswith(f"tcc-uema-app-{s}") and not
(s == "a" and "nginx-a" in r["name"]):
                continue
            cpu = float(r["cpu_pct"].rstrip("%"))
            mem = r["mem_usage"].split("/")[0].strip()
            val = float(re.sub("[^0-9.]", "", mem)) * (1024 if "GiB" in mem else 1)
            c, m = por_ts.get(ts, (0, 0))
            por_ts[ts] = (c + cpu, m + val)
        if por_ts:
            cs = [v[0] for v in por_ts.values()]
            ms_ = [v[1] for v in por_ts.values()]
            res[s].append({"cpu": st.mean(cs), "ram": st.mean(ms_), "ram_pico": max(ms_)})
    return res



def ms(vals):
    vals = [v for v in vals if v == v]
    if not vals:
        return (float("nan"), float("nan"))
    return (st.mean(vals), st.stdev(vals) if len(vals) > 1 else 0.0)



def main():
    data = load()
    agg = {}
    for s, runs in data.items():
        if not runs:
            continue
        agg[s] = {k: ms([r[k] for r in runs if k in r]) for k in runs[0]}
        agg[s]["n"] = len(runs)

    # Tabela 1/2 (CSV + Markdown)
    linhas = ["| Cenário | n | RPS (média ± dp) | Latência p50 (ms) | p95 (ms) | p99 (ms) |
Erros (%) |",
              "|---|---|---|---|---|---|---|"]
    with open(OUT / "global.csv", "w", newline="") as f:
```

```
        w = csv.writer(f)
        w.writerow(["cenario", "n", "rps_media", "rps_dp", "p50", "p95", "p99", "erro_pct"])
        for s, a in agg.items():
            linhas.append(
                f"| {NOMES[s]} | {a['n']} | {a['rps'][0]:.1f} ± {a['rps'][1]:.1f} | {a['p50']
[0]:.1f} | "
                f"{a['p95'][0]:.1f} | {a['p99'][0]:.1f} | {a['err'][0]*100:.2f} |")
            w.writerow([s, a["n"], a["rps"][0], a["rps"][1], a["p50"][0], a["p95"][0],
a["p99"][0], a["err"][0] * 100])
    (OUT / "tabela_global.md").write_text("\n".join(linhas) + "\n")

    linhas = ["| Cenário | " + " | ".join(f"{e} avg / p95 (ms)" for e in ENDPOINTS) + " |",
              "|---|" + "---|" * len(ENDPOINTS)]
    for s, a in agg.items():
        cel = [f"{a[f'{e}_avg'][0]:.1f} / {a[f'{e}_p95'][0]:.1f}" for e in ENDPOINTS]
        linhas.append(f"| {NOMES[s]} | " + " | ".join(cel) + " |")
    (OUT / "tabela_endpoints.md").write_text("\n".join(linhas) + "\n")

    # Tabela 3: recursos
    rec = recursos()
    linhas = ["| Cenário | CPU média (%) | RAM média (MiB) | RAM pico (MiB) |",
"|---|---|---|---|"]
    ragg = {}
    for s, rr in rec.items():
        if rr:
            ragg[s] = {k: ms([r[k] for r in rr]) for k in ("cpu", "ram", "ram_pico")}
            a = ragg[s]
            linhas.append(f"| {NOMES[s]} | {a['cpu'][0]:.1f} ± {a['cpu'][1]:.1f} | {a['ram']
[0]:.1f} ± {a['ram'][1]:.1f} | {a['ram_pico'][0]:.1f} |")
    (OUT / "tabela_recursos.md").write_text("\n".join(linhas) + "\n")

    # Ganhos relativos de RPS
    ganhos = []
    for a_, b_ in [("b", "a"), ("c", "a"), ("c", "b"), ("c", "d"), ("d", "a")]:
        if a_ in agg and b_ in agg:
            g = (agg[a_]["rps"][0] / agg[b_]["rps"][0] - 1) * 100
            ganhos.append(f"{a_.upper()} vs {b_.upper()}: {g:+.1f}% RPS")
    (OUT / "ganhos.txt").write_text("\n".join(ganhos) + "\n")

    # Figuras
    try:
        import matplotlib
        matplotlib.use("Agg")
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib ausente: figuras não geradas")
        return
    plt.rcParams.update({"font.size": 11, "font.family": "DejaVu Sans"})

    ss = list(agg)
    fig, ax = plt.subplots(figsize=(7, 4))
    ax.bar([NOMES[s].split(" — ")[0] for s in ss], [agg[s]["rps"][0] for s in ss],
           yerr=[agg[s]["rps"][1] for s in ss], color=[CORES[s] for s in ss], capsize=4)
    for i, s in enumerate(ss):
        ax.text(i, agg[s]["rps"][0], f"{agg[s]['rps'][0]:.0f}", ha="center", va="bottom")
    ax.set_ylabel("Requisições por segundo (média ± dp)")
    ax.set_xlabel("Cenário")
    fig.tight_layout()
    fig.savefig(OUT / "fig_rps.png", dpi=200)
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(8, 4.2))
    w = 0.8 / len(ss)
    for i, s in enumerate(ss):
        ax.bar([j + i * w for j in range(len(ENDPOINTS))], [agg[s][f"{e}_p95"][0] for e in
ENDPOINTS],
               w, label=NOMES[s].split(" — ")[0], color=CORES[s])
    ax.set_xticks([j + 0.4 - w / 2 for j in range(len(ENDPOINTS))])
    ax.set_xticklabels(ENDPOINTS)
    ax.set_ylabel("Latência p95 (ms)")
    ax.set_xlabel("Endpoint")
    ax.legend(title="Cenário")
    fig.tight_layout()
    fig.savefig(OUT / "fig_latencia_p95.png", dpi=200)
```

```
    plt.close(fig)
    if ragg:
        fig, axs = plt.subplots(1, 2, figsize=(9, 3.8))
        ks = list(ragg)
        for ax, k, t in ((axs[0], "cpu", "CPU média (%)"), (axs[1], "ram", "RAM média
(MiB)")):
            ax.bar([NOMES[s].split(" — ")[0] for s in ks], [ragg[s][k][0] for s in ks],
                   yerr=[ragg[s][k][1] for s in ks], color=[CORES[s] for s in ks], capsize=4)
            ax.set_ylabel(t)
            ax.set_xlabel("Cenário")
        fig.tight_layout()
        fig.savefig(OUT / "fig_recursos.png", dpi=200)
        plt.close(fig)
    print("OK:", OUT)


if __name__ == "__main__":
    main()
```

APÊNDICE E — CONTROLADOR DOS ENDPOINTS (CODEIGNITER 4)

apps/ci4-base/app/Controllers/BenchmarkController.php

```
<?php

namespace App\Controllers;

use CodeIgniter\Controller;

class BenchmarkController extends Controller
{
    public function health(): void
    {
        $this->response->setHeader('Content-Type', 'application/json');
        echo json_encode([
            'status'   => 'ok',
            'scenario' => getenv('SCENARIO_NAME') ?: 'unknown',
        ]);
    }

    public function compute(): void
    {
        $size = (int) (getenv('COMPUTE_SIZE') ?: 1000);
        $data = [];

        for ($i = 0; $i < $size; $i++) {
            $data[] = [
                'id'    => $i + 1,
                'uuid'  => md5(uniqid((string) $i, true)),
                'value' => random_int(1, 999999),
                'label' => str_repeat('x', random_int(5, 20)),
            ];
        }

        $this->response->setHeader('Content-Type', 'application/json');
        echo json_encode([
            'count'    => $size,
            'scenario' => getenv('SCENARIO_NAME') ?: 'unknown',
            'data'     => $data,
        ]);
    }

    public function memory(): void
    {
        // Propriedade estática sobrevive ao loop do Worker Mode (mesma instância PHP)
        // Em PHP-FPM/Classic: reinicia a cada request (processo novo ou sem estado
persistente)
        MemoryState::$counter++;

        $this->response->setHeader('Content-Type', 'application/json');
        echo json_encode([
            'counter'  => MemoryState::$counter,
            'pid'      => getmypid(),
            'scenario' => getenv('SCENARIO_NAME') ?: 'unknown',
        ]);
    }

    public function db(): void
    {
        $id = random_int(1, 10000);

        // pg_backend_pid() retorna o PID do processo PostgreSQL que serve esta conexão.
        // Worker Mode (C, D): pg_pid constante = mesma conexão reutilizada entre requests.
        // PHP-FPM/Classic (A, B): pg_pid varia = nova conexão estabelecida por request.
        // Incluído na mesma query — sem round-trip extra.
        $row = db_connect()
            ->query(
                'SELECT id, name, value, pg_backend_pid() AS pg_pid FROM benchmark_items WHERE
id = ?',
```

```
                [$id]
            )
            ->getRow();

        $this->response->setHeader('Content-Type', 'application/json');
        echo json_encode([
            'item'     => $row,
            'php_pid'  => getmypid(),
            'scenario' => getenv('SCENARIO_NAME') ?: 'unknown',
        ]);
    }
}

class MemoryState
{
    public static int $counter = 0;
}
```

APÊNDICE F — RESULTADOS POR EXECUÇÃO

Tabela F1 – Resultados de cada uma das 12 execuções

Execução	RPS	Latência p50 (ms)	Latência p95 (ms)	Erros (%)
A – execução 1	124.9	786.4	896.9	0.00
A – execução 2	126.5	783.4	890.4	0.00
A – execução 3	127.8	723.8	885.4	0.00
B – execução 1	109.2	881.7	1005.9	0.00
B – execução 2	109.2	878.6	1002.2	0.00
B – execução 3	110.1	873.0	998.4	0.00
C – execução 1	291.5	307.6	399.7	0.00
C – execução 2	289.8	308.4	399.7	0.00
C – execução 3	281.9	307.6	493.0	0.00
D – execução 1	368.6	277.2	313.6	0.00
D – execução 2	366.6	277.6	318.4	0.00
D – execução 3	370.8	272.7	312.3	0.00

Fonte: elaborado pelo autor (2026).